

TRIBHUVAN UNIVERSITY
Institute of Science and Technology
[COLLEGE/CAMPUS NAME]

A FINAL YEAR PROJECT REPORT

on

**“OMR-Web: A Web-Based Optical Mark Recognition System for
Automated Answer Sheet Grading”**

Submitted to
Department of Computer Science and Information Technology
[COLLEGE/CAMPUS NAME]

In partial fulfillment of the requirements for the degree of Bachelor of Science in
Computer Science and Information Technology (B.Sc. CSIT)

Submitted by
[STUDENT NAME]
Symbol No.: [SYMBOL NO.]
T.U. Registration No.: [T.U. REGISTRATION NO.]

Under the Supervision of
[SUPERVISOR NAME]

[Month, Year]

SUPERVISOR RECOMMENDATION

This is to recommend that [STUDENT NAME] (Symbol No. [SYMBOL NO.], Registration No. [T.U. REGISTRATION NO.]) has carried out the project work entitled “OMR-Web: A Web-Based Optical Mark Recognition System for Automated Answer Sheet Grading” for the requirement of the project work in Bachelor of Science (B.Sc.) degree in Computer Science and Information Technology under my supervision in the Department of Computer Science and Information Technology (CSIT), [COLLEGE/CAMPUS NAME], Institute of Science and Technology (IoST), Tribhuvan University, Nepal.

To my knowledge, this work has not been submitted for any other degree. The student has fulfilled all the requirements laid down by the Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the submission of the project work for the partial fulfillment of the Bachelor of Science in Computer Science and Information Technology (B.Sc. CSIT) degree.

SIGNATURE

.....

[SUPERVISOR NAME]

Supervisor

Department of Computer Science and Information Technology

[COLLEGE/CAMPUS NAME]

[Month, Year]

CERTIFICATE OF APPROVAL

This is to certify that the report is evaluated and recommended to the Department of Computer Science and Information Technology for the acceptance of the report entitled “OMR-Web: A Web-Based Optical Mark Recognition System for Automated Answer Sheet Grading” submitted by [STUDENT NAME] in partial fulfillment for the degree of Bachelor of Science in Computer Science and Information Technology (B.Sc. CSIT), Institute of Science and Technology (IoST), Tribhuvan University.

SIGNATURE of Supervisor	SIGNATURE of Coordinator
[SUPERVISOR NAME] Supervisor Department of CSIT [COLLEGE/CAMPUS NAME] [College Address]	[HOD/COORDINATOR NAME] Coordinator of CSIT Department of CSIT [COLLEGE/CAMPUS NAME] [College Address]
SIGNATURE of Internal Examiner Internal Examiner	SIGNATURE of External Examiner External Examiner

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my college for providing a suitable platform to enhance my knowledge and demonstrate my skills through this final year project. Working on this project has been a valuable learning experience, allowing me to apply theoretical concepts of image processing, computer vision, and web application development in a practical manner.

I am deeply thankful to my respected supervisor, [SUPERVISOR NAME], for the invaluable guidance, continuous encouragement, and constructive feedback provided throughout the development of this project. The regular technical discussions and reviews were instrumental in shaping this project into its final form.

I am also grateful to all the faculty members of the Department of Computer Science and Information Technology for their guidance during my academic journey, and to my friends and classmates for their cooperation and encouragement. Finally, I extend my appreciation to everyone who directly or indirectly contributed to the successful completion of this project.

With respect,

[STUDENT NAME]

Symbol No.: [SYMBOL NO.]

T.U. Registration No.: [T.U. REGISTRATION NO.]

ABSTRACT

Evaluating multiple-choice examinations by hand is slow, repetitive, and error-prone, while dedicated Optical Mark Recognition (OMR) machines and pre-printed sheets are expensive for most institutions. This project, OMR-Web, is a web-based OMR system that grades photographed or scanned bubble-sheet answer sheets using classical computer vision, without any special hardware. An examiner uploads an answer key (CSV), a sheet template describing bubble positions, and a batch of sheet images through a browser interface. For every image, the system locates four circular registration marks, applies a perspective transformation to deskew the sheet to a canonical size, and automatically corrects upside-down captures. The corrected sheet is binarized, and an integral image is used to compute the fill ratio of every roll-number and answer bubble in a single vectorized pass, which makes grading fast enough for live feedback. Decoded responses are scored against the answer key with support for per-question marks, negative marking, and subject-wise totals. Grading runs as a background job; the browser polls for progress and displays annotated previews of each graded sheet, and the final results are exported as a CSV report. The platform is delivered as a role-based web application built with Next.js and Clerk authentication: administrators run grading batches and study an analytics dashboard of the results (score distribution, subject strength, hardest questions, per-student table), students sign in to look up only their own scorecard and compare it with the batch, and a super administrator holds every permission. In testing on a batch of 80 photographed sheets containing 100 questions each, the system calibrated and graded every sheet at approximately 6.5 sheets per second, demonstrating that an inexpensive, software-only OMR pipeline can deliver accurate and practical automated grading.

KEYWORDS: *Optical Mark Recognition, Computer Vision, Image Processing, Perspective Transformation, Integral Image, FastAPI, Next.js, Role-Based Access Control, Automated Grading*

Table of Contents

Right-click here and choose “Update Field” to generate the Table of Contents.

LIST OF ABBREVIATIONS

API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CSIT	Computer Science and Information Technology
CSV	Comma-Separated Values
CV	Computer Vision
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPEG	Joint Photographic Experts Group
JS	JavaScript
JSON	JavaScript Object Notation
MCQ	Multiple Choice Question
OMR	Optical Mark Recognition
PNG	Portable Network Graphics
REST	Representational State Transfer
ROI	Region of Interest
UI	User Interface
UML	Unified Modeling Language
UUID	Universally Unique Identifier

LIST OF FIGURES

Figure 1.1: Incremental Delivery Model.....	
Figure 3.1: Use Case Diagram of the OMR-Web System.....	
Figure 3.2: Gantt Chart.....	
Figure 3.3: Class Diagram.....	
Figure 3.4: Object Diagram.....	
Figure 3.5: Sequence Diagram of the Grading Workflow.....	
Figure 3.6: Activity Diagram of the Grading Pipeline.....	
Figure 4.1: High-Level Design of the System.....	
Figure 4.2: Refined Class Diagram.....	
Figure 4.3: Refined Sequence Diagram.....	
Figure 4.4: Refined Activity Diagram.....	
Figure 4.5: Component Diagram.....	
Figure 4.6: Deployment Diagram.....	
Figure 5.1: Login Page (Clerk Authentication).....	
Figure 5.2: Admin Dashboard Before Any Job (Empty State).....	
Figure 5.3: Upload Hub with Sheets, Answer Key, and Template Selected.....	
Figure 5.4: Live Grading in Progress (Job Card with Annotated Preview).....	
Figure 5.5: Exam Analytics Dashboard (KPIs, Charts, and Student Table).....	
Figure 5.6: Checked-Sheets Review (Split-Pane with Per-Question Breakdown).....	
Figure 5.7: Student Portal — Personal Scorecard and Own Sheet Review.....	
Figure 5.8: Super Admin Viewing a Batch Loaded by Job ID.....	
Figure 5.9: Auto-Generated API Documentation (Swagger UI, /docs).....	

Note: page numbers are inserted after final pagination in Word.

LIST OF TABLES

Table 3.1: Schedule Table.....	
Table 5.1: REST API Endpoints of the System.....	
Table 5.2: User Roles and Access in the Platform.....	
Table 5.3: Test Cases for Answer Key and Upload Validation.....	
Table 5.4: Test Cases for Bubble Detection and Decoding.....	
Table 5.5: Test Cases for System Testing.....	
Table 5.6: Measured Performance on the Test Batch.....	

Note: page numbers are inserted after final pagination in Word.

CHAPTER 1

INTRODUCTION

1.1 Introduction

Multiple-choice examinations remain one of the most widely used assessment formats in schools, colleges, and competitive entrance tests because they are objective and quick for students to answer. Evaluating them, however, is a different matter: an examiner checking hundreds of answer sheets by hand must compare every response against an answer key, apply positive and negative marking, total the score, and transcribe the result — a process that is slow, monotonous, and prone to human error. Institutions that can afford them use dedicated Optical Mark Recognition (OMR) machines, which read specially printed sheets using optical sensors. Such machines and their proprietary pre-printed sheets are costly, which puts automated grading out of reach for many institutions.

OMR-Web addresses this gap with a purely software-based approach. It is a web application that grades ordinary photographed or scanned bubble sheets using classical computer vision techniques. The examiner uploads three inputs through a browser: an answer key in CSV format, a template file that describes where every bubble lies on the sheet, and a batch of sheet images. The backend then calibrates each photograph against the template by locating four circular registration marks printed on the sheet corners, applies a perspective transformation to remove skew and scale differences, and automatically corrects sheets that were photographed upside down. The corrected sheet is thresholded into a binary image, and an integral image is used to measure how much ink fills each bubble region in one vectorized computation, rather than examining bubbles one at a time. Bubbles whose fill ratio exceeds a threshold are treated as marked; each question is then decoded as a selected option, a skipped question, or an invalid response when multiple options are marked.

Decoded answers are scored against the uploaded answer key, which specifies per-question marks, the negative-marking percentage, and the subject of each question, so the system produces both an overall score and subject-wise totals. Grading runs as a background job on the server: the browser polls a progress endpoint and displays a live, annotated preview of every graded sheet, on which each detected mark is boxed and

labelled as correct, incorrect, or invalid, with the decoded roll number and score overlaid. When the job completes, the examiner can page through every checked sheet and download the full result as a CSV report.

The system is delivered as a two-tier web platform. A FastAPI backend hosts the OpenCV-based grading engine, and a Next.js (React) frontend delivers a role-based user experience secured by Clerk authentication. Three roles are supported: an organisation administrator (the examiner) runs grading batches, monitors live progress, studies an analytics dashboard of the batch — passing rate, score distribution, subject strength, and the hardest questions — and reviews every checked sheet; a student signs in to look up only their own scorecard, rank, and annotated sheet; and a super administrator passes every access guard for full oversight of the platform.

1.2 Problem Statement

Manual evaluation of multiple-choice answer sheets suffers from several well-known problems. It is slow: a single examiner may need several minutes per sheet once negative marking and subject-wise totals are involved. It is error-prone: fatigue leads to misread marks, wrong key lookups, and arithmetic mistakes, and such errors are difficult to audit after the fact. It also scales poorly: as the number of candidates grows, the only manual remedy is more examiners and more time.

Dedicated OMR machines solve the speed problem but introduce new ones. The hardware is expensive to purchase and maintain, it typically requires proprietary pre-printed sheets with precise ink specifications, and it offers little flexibility when the sheet layout, marking scheme, or number of questions changes. Existing software alternatives often assume flatbed-scanned images, fail on photographs taken at an angle or rotated, or provide no usable interface for a non-technical examiner.

Therefore, there is a clear need for a system that can:

- Grade ordinary photographed or scanned bubble sheets accurately without any dedicated OMR hardware.
- Tolerate real-world capture conditions such as perspective skew, scale differences, and 180° rotation.
- Apply a configurable marking scheme with per-question marks, negative marking, and subject-wise totals.

- Give the examiner immediate visual feedback for every graded sheet so results can be verified and disputed marks inspected.
- Export results in a standard format (CSV) usable by existing record-keeping workflows.

The OMR-Web project addresses these problems by combining marker-based image calibration, integral-image bubble detection, and a lightweight web interface into a single, hardware-free grading pipeline.

1.3 Objectives

The main goal of this project is to develop a reliable, low-cost, web-based OMR system for automated grading of bubble-sheet examinations. The specific objectives are:

- To develop a computer-vision pipeline that calibrates photographed answer sheets using corner registration marks and detects filled bubbles using integral-image fill-ratio analysis, and scores the decoded responses against a configurable answer key supporting negative marking and subject-wise totals.
- To build a role-based web platform where an administrator can run grading batches with live progress and annotated previews, study batch analytics, and export results as CSV, while an authenticated student can securely look up only their own scorecard and compare it with the batch.

1.4 Scope and Limitation

The scope of OMR-Web covers the complete grading workflow for single-choice, four-option (A–D) multiple-choice answer sheets. The system accepts batches of sheet photographs or scans, calibrates each image against a template describing the canonical bubble layout, decodes the candidate's roll number from a bubble grid, grades every question with configurable positive and negative marking, accumulates subject-wise totals, renders an annotated preview of every sheet for verification, and exports a consolidated CSV report. Grading executes asynchronously on the server so that large batches can be processed while the examiner watches live progress. Sheets that cannot be calibrated are preserved separately for manual review rather than silently discarded. Access to the platform is role-based: users authenticate through Clerk, and route-level guards derive each user's role (SUPER_ADMIN, ORG_ADMIN, or STUDENT) from the

session, giving administrators the grading and analytics workflow and students a read-only lookup of their own result.

The limitations of the project include:

- **Template dependency:** bubble positions are read from a template file produced by a separate template-builder tool; the system cannot infer an unknown sheet layout on its own.
- **Fixed thresholding:** binarization uses a fixed intensity threshold, so extremely poor or uneven lighting can affect mark detection.
- **Marker requirement:** calibration requires all four circular corner registration marks to be visible; sheets with torn or covered corners are rejected as invalid.
- **Single-choice questions only:** multiple marks on one question are treated as invalid; multiple-answer question formats are not supported.
- **In-memory job storage:** job state lives in server memory, so grading history is lost when the server restarts and results should be exported promptly.

1.5 Development Methodology

The project was developed using incremental delivery, a form of iterative software development in which the system is built as a series of increments, each adding a working capability on top of the previous ones. This approach suited the project because the computer-vision pipeline needed continuous experimentation: parameters such as the binarization threshold, the bubble-region size, and the fill-ratio cut-off could only be tuned by grading real sheet photographs and inspecting the results.

In the first increment, the core calibration module was developed and validated: detecting the four corner registration marks, ordering them, and warping the photograph to the template's canonical size. The second increment implemented bubble detection and decoding using integral images, first for the roll-number grid and then for the answer grid. The third increment added scoring against the CSV answer key, including negative marking and subject-wise accumulation. The fourth increment wrapped the engine in a FastAPI web service with background job execution, progress reporting, and preview streaming, and the final increment delivered the browser frontend, the checked-sheets review page, and CSV report export. Each increment was tested against a batch of real photographed sheets before the next was started, allowing detection errors to be found and fixed early.

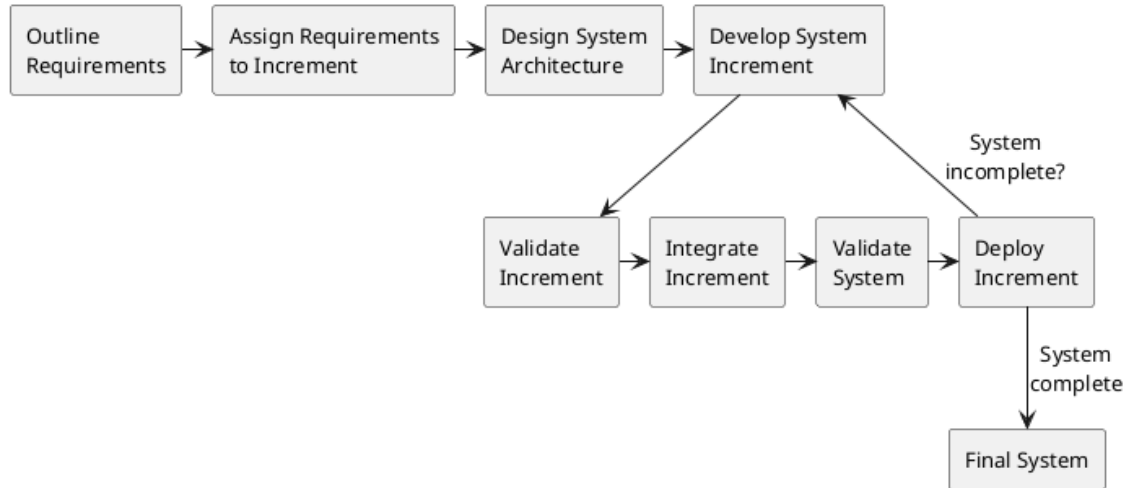


Figure 1.1: Incremental Delivery Model

1.6 Report Organization

This report is organized into six chapters, each describing a distinct stage of the system's development.

Chapter 1 introduces the project, presenting the background, problem statement, objectives, scope and limitations, and the development methodology followed.

Chapter 2 provides the background study and literature review, covering the fundamental image-processing concepts used by the system and reviewing related work on optical mark recognition.

Chapter 3 focuses on system analysis, including functional and non-functional requirements, feasibility analysis, and object-oriented analysis using class, object, sequence, and activity diagrams.

Chapter 4 describes the system design: the high-level processing pipeline, refined UML diagrams, component and deployment views, and a detailed description of the algorithms used for calibration, bubble detection, and scoring.

Chapter 5 covers implementation and testing, detailing the tools used, the implementation of each module, unit and system test cases, and an analysis of the measured results.

Chapter 6 concludes the report and proposes future enhancements to the system.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Optical Mark Recognition is the technique of detecting deliberate human-made marks — typically filled bubbles or checkboxes — on a printed form. Classical OMR, introduced with dedicated scanning machines, senses the reflectance of light at fixed positions on specially printed sheets. Image-based OMR, the approach used in this project, instead operates on an ordinary digital image of the sheet and must therefore solve two problems that dedicated hardware avoids: locating the sheet's coordinate system inside an arbitrary photograph, and deciding robustly whether each bubble region is filled.

The first problem is one of geometric calibration. A photographed sheet appears at an unknown position, scale, rotation, and perspective. If the sheet carries known fiducial markers — in this project, four solid circles printed near the corners — the distortion can be undone with a planar perspective transformation (homography). The four detected marker centres are put into correspondence with the four corners of the template's canonical rectangle, a 3×3 transformation matrix is estimated, and the whole image is warped so that every bubble lands at the pixel coordinates recorded in the template [6]. Contour detection, used here to find the markers, follows the classical border-following algorithm of Suzuki and Abe [3], as implemented in OpenCV. Because a sheet can also be photographed upside down (a 180° rotation maps a rectangle onto itself), the system disambiguates orientation by comparing ink density near the top and bottom marker midpoints.

The second problem is mark detection. After calibration, the grayscale sheet is reduced to a binary image with intensity thresholding, a standard segmentation technique [4]: pixels darker than a threshold become ink (1) and the rest background (0). Deciding whether a bubble is filled then reduces to summing the ink pixels inside its region. Summing pixel-by-pixel for hundreds of bubbles per sheet is wasteful; instead the system computes an integral image (summed-area table), popularized in computer vision by Viola and Jones [2], in which each entry holds the sum of all pixels above and to the left of it. Any rectangular region's sum is then obtained with just four array lookups, independent of the region's size. The system refines this further by summing over an ellipse inscribed in each

bubble's bounding box — matching the printed bubble's shape — by accumulating one rectangular strip per pixel row of the ellipse. The resulting ink sum divided by the ellipse's area gives a fill ratio between 0 and 1 that can be compared against a single tuned threshold.

The delivery vehicle for the pipeline is a web application. The backend uses FastAPI [8], a modern Python web framework, with grading executed on a background thread so that the HTTP interface stays responsive; the browser polls lightweight JSON endpoints for progress and fetches JPEG previews as sheets are graded. All image processing uses the OpenCV library [5] with NumPy for vectorized array computation. The frontend is a Next.js (React) application, and Clerk provides hosted authentication with per-user role metadata that server-side middleware reads to guard every route — separating what an administrator, a student, and a super administrator can see and do.

2.2 Literature Review

Optical mark recognition has been studied both as a hardware problem and, more recently, as a computer-vision problem. de Elias et al. [1] survey the advances, difficulties, and limitations of OMR systems, noting that while dedicated scanners achieve high accuracy, their cost and inflexibility have driven research toward camera-based and software-only solutions, whose main challenges are geometric distortion, uneven illumination, and unreliable mark/no-mark discrimination — precisely the challenges this project's calibration and fill-ratio design addresses.

Sattayakawee [9] demonstrated grading of non-optical grid answer sheets using projection profile analysis on thresholded scans, achieving high accuracy on flatbed-scanned sheets but assuming an upright, unskewed input. Marker-based approaches remove that assumption: systems that print fiducial shapes on the sheet and estimate a homography from their detected positions can grade photographs taken at an angle. The open-source OMRChecker project [7] popularized this style of template-driven, camera-based OMR in Python and showed that classical vision techniques, without any machine learning, are sufficient for production-quality grading when calibration is reliable.

For the mark-detection step itself, thresholding followed by regional ink measurement is the dominant classical technique. Otsu's method [4] selects a global threshold automatically from the image histogram; this project instead uses a fixed threshold tuned for its sheet stock and printing, trading some illumination robustness for predictability.

The integral image used to accelerate regional sums was introduced to real-time vision by Viola and Jones [2] for face detection, where thousands of rectangular features must be evaluated per frame — the same computational pattern as evaluating hundreds of bubbles per sheet, which is why it transfers so effectively to OMR.

In summary, the reviewed literature indicates that classical, interpretable image processing — fiducial-based homography calibration, binary thresholding, and integral-image region analysis — remains a strong and computationally efficient foundation for automated sheet grading, particularly for systems that must run in real time on modest hardware and whose decisions must be visually verifiable by an examiner. OMR-Web builds directly on these results and packages them behind a web interface aimed at non-technical users.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Analysis

System analysis breaks the proposed system into its components and interactions to establish what must be built before design begins. For OMR-Web, the analysis identifies the grading workflow from the examiner's perspective, the requirements the computer-vision pipeline must satisfy, the feasibility of building the system with open-source tools within one semester, and object-oriented models of the system's structure and behaviour.

3.1.1 Requirement Analysis

i. Functional Requirements

The main functional requirements of the OMR-Web system are as follows:

- **Authentication and role-based access:** users shall sign in through Clerk; route guards shall admit SUPER_ADMIN everywhere, ORG_ADMIN to the admin workspace, and STUDENT to the student portal, and each signed-in user shall be routed to their role's home page automatically.
- **Upload management:** the system shall let the examiner upload an answer key CSV, a template file, and one or more sheet images from the browser in a single form submission.
- **Upload validation:** the system shall validate the answer key (required columns Question, Subject, Marks_Correct, Negative_Percent, Answer; numeric, non-negative marks) and the template file before any grading starts, and reject invalid uploads with a clear error message.
- **Background grading job:** the system shall grade the batch on a background worker identified by a unique job ID, so the browser is never blocked while sheets are processed.
- **Sheet calibration:** the system shall locate the four corner registration marks in each photograph, perspective-correct the sheet to the template's canonical size, and automatically fix 180°-rotated captures.

- **Mark detection and decoding:** the system shall measure the fill ratio of every roll-number and answer bubble, decode each question as a selected option (A–D), skipped, or invalid (multiple marks), and decode the roll number from its bubble grid.
- **Scoring:** the system shall score each sheet against the answer key, applying per-question marks and negative marking, and accumulate subject-wise totals alongside the overall score.
- **Live progress and preview:** the system shall report progress (sheets done, rate, estimated time remaining) and serve an annotated preview image of each graded sheet, with detected marks boxed and labelled correct, incorrect, or invalid.
- **Checked-sheet review:** the system shall provide a review page where the examiner can page through every graded sheet of a job.
- **Report export:** the system shall export a CSV report containing, for every sheet, the file name, roll number, per-question responses, subject-wise marks, counts of correct/wrong/skipped/invalid answers, and the total score.
- **Invalid sheet handling:** sheets that cannot be calibrated shall be saved to an Invalid_sheets folder for manual review and reported as errors rather than silently skipped.
- **Analytics dashboard:** for a completed batch the system shall present students graded, passing rate, class average, invalid-sheet count, a score distribution, subject-wise strength, the hardest questions by incorrect-answer rate, and a searchable, filterable student table with per-sheet drill-down.
- **Student result lookup:** a student shall retrieve only their own result using the exam's job id and their roll number, seeing their score, rank, batch comparison, response counts, and their own annotated sheet.

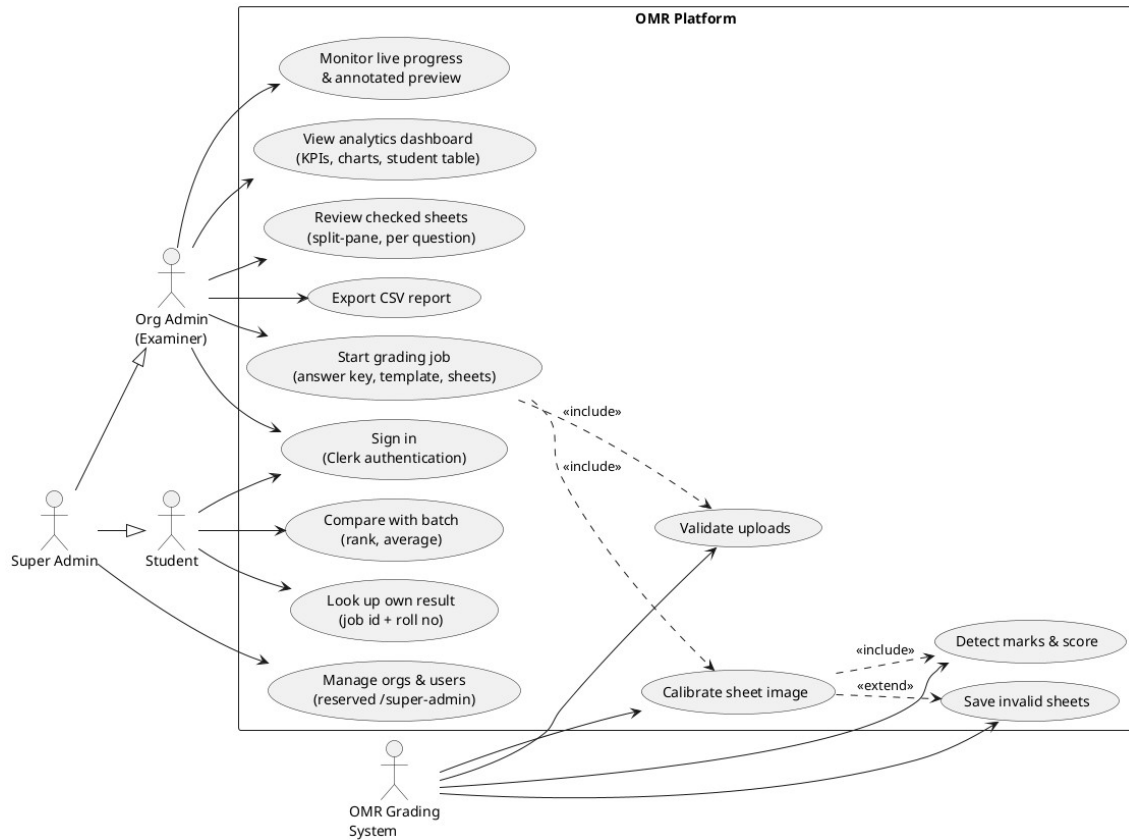


Figure 3.1: Use Case Diagram of the OMR Platform

The use case diagram shows the three human actors. The Org Admin (examiner) signs in and drives the grading workflow — starting jobs, monitoring progress, studying analytics, reviewing sheets, and exporting reports. The Student signs in to look up and compare only their own result. The Super Admin inherits both roles' capabilities (shown by the actor generalization) and additionally owns the reserved organisation-management area. The system-side use cases — validation, calibration, mark detection and scoring, and invalid sheet preservation — are included by starting a grading job.

ii. Non-Functional Requirements

- **Performance:** grading must be fast enough for interactive use; the vectorized integral-image design targets several sheets per second on an ordinary laptop, and progress updates must be cheap enough to poll every 300 milliseconds.
- **Usability:** the interface must be operable by a non-technical examiner: three file pickers, one run button, a visible progress bar, and previews that make every grading decision visually verifiable.

- **Reliability:** a single bad image must not corrupt a batch: uploads are validated before the job starts, undecodable images are skipped, and calibration failures are isolated and reported per file.
- **Security:** every dashboard route must require an authenticated session; role checks must run in server-side middleware on each request, so a student cannot reach admin pages by typing a URL, and authentication secrets must never reach the browser.
- **Portability:** the system must run anywhere Python, Node.js, and OpenCV run, with no dedicated hardware and no database server.

3.1.2 Feasibility Analysis

a. Technical Feasibility

The technical feasibility of the project is high, since every required tool is free, mature, and well documented.

Hardware Requirements:

- A computer with at least 8 GB RAM and a multi-core processor.
- An ordinary smartphone camera or scanner to capture the answer sheets.
- Adequate disk space for sheet images and exported reports.

Software Requirements:

- Operating System: Linux or Windows.
- Backend: Python 3.12+, FastAPI framework, Uvicorn ASGI server.
- Image processing: OpenCV and NumPy.
- Frontend: Node.js/Bun runtime, Next.js 14 with React 18 and TypeScript, Tailwind CSS.
- Authentication: Clerk (hosted user management, sessions, and per-user role metadata).
- Version control and quality tooling: Git/GitHub, with lefthook, oxlint, and prettier for pre-commit checks.

b. Operational Feasibility

Operational feasibility is high. The examiner's workflow — photograph the sheets, upload three files, watch progress, download a CSV — mirrors familiar web-form usage and requires no training in image processing. Because every graded sheet is returned as an annotated preview, the examiner can audit any individual grading decision, which builds

the trust necessary for the system to actually replace manual checking. The main operational dependency is capture quality: all four corner markers must be visible in the photograph, which is easy to satisfy with minimal instruction.

c. Economic Feasibility

The project is economically feasible because it is built entirely from open-source software — Python, FastAPI, OpenCV, and NumPy are all free — and eliminates the two costs that dominate traditional OMR: the scanning machine and proprietary pre-printed sheets. Sheets can be printed on ordinary paper with a normal printer, and capture uses devices the institution already owns. Beyond a standard computer, the system has no acquisition or licensing cost, and maintenance is limited to ordinary software upkeep.

d. Schedule Feasibility

The project was planned over one semester. The computer-vision modules (calibration and the OMR engine) were allotted the largest share of time because their parameters required iterative tuning against real photographs. The schedule below proved achievable, making the project schedule feasible.

Table 3.1: Schedule Table

Phase	Duration	Start Date	End Date
1. Planning & Requirement Gathering	7 days	1 Mar 2026	7 Mar 2026
2. System Analysis & Design	14 days	8 Mar 2026	21 Mar 2026
3. Calibration Module Development	14 days	22 Mar 2026	4 Apr 2026
4. OMR Engine Development	21 days	5 Apr 2026	25 Apr 2026
5. Web API & Job Management	21 days	26 Apr 2026	16 May 2026
6. Frontend Design & Integration	14 days	17 May 2026	30 May 2026
7. Testing & Debugging	14 days	31 May 2026	13 Jun 2026
8. Documentation & Report Writing	14 days	14 Jun 2026	27 Jun 2026
9. Final Review & Presentation	7 days	28 Jun 2026	4 Jul 2026

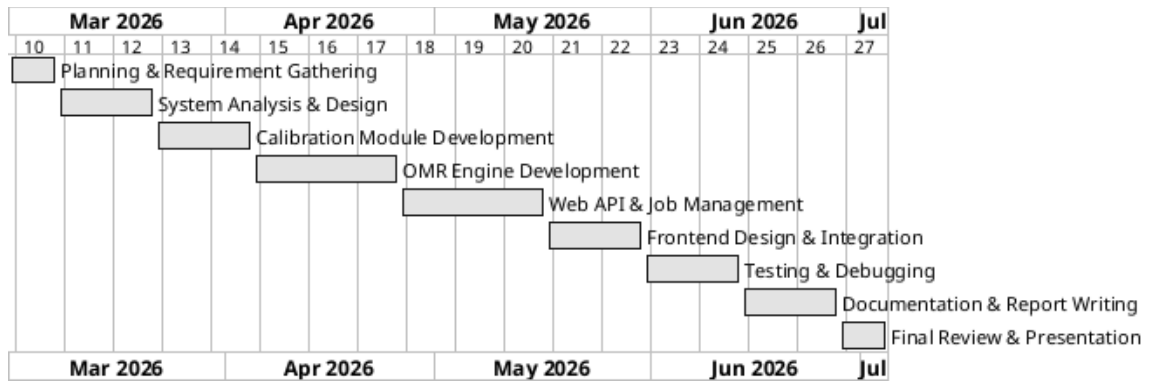


Figure 3.2: Gantt Chart

3.1.3 Analysis

The system was analysed using the object-oriented approach, modelling its static structure with class and object diagrams and its dynamic behaviour with sequence and activity diagrams.

Object Modeling using Class Diagram

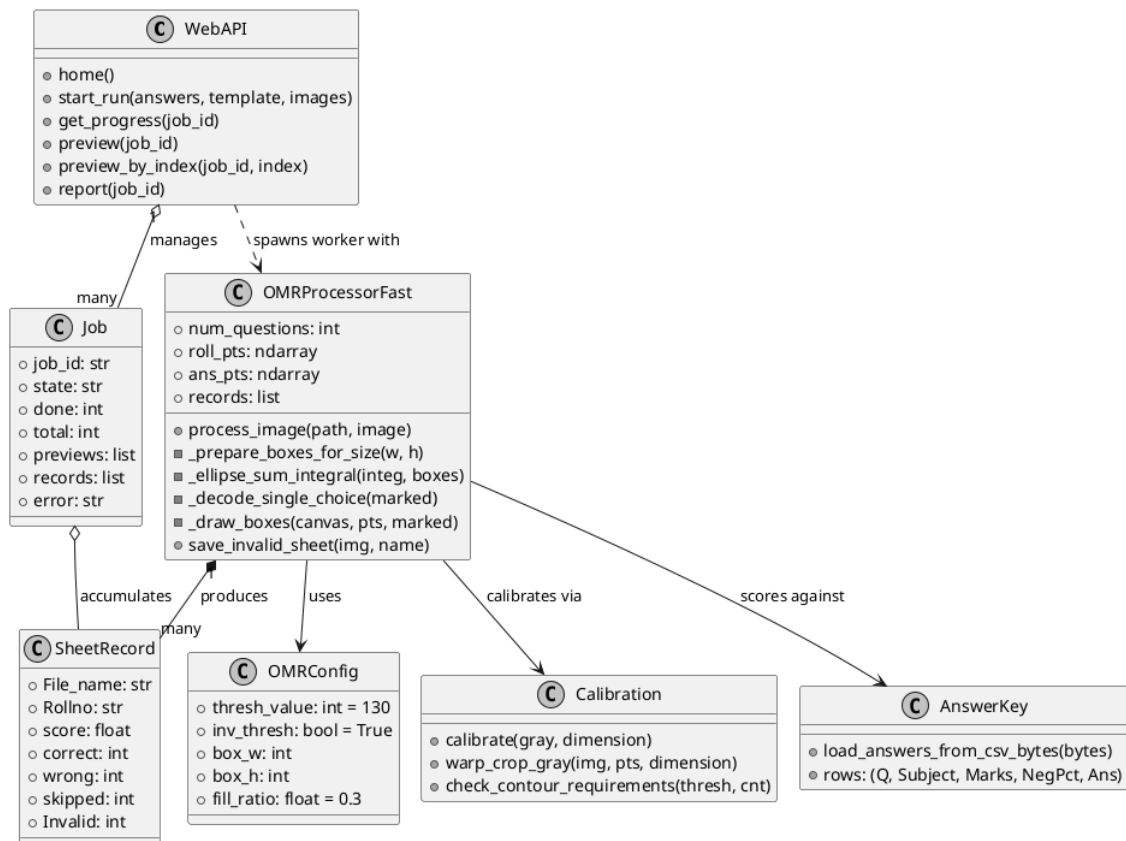


Figure 3.3: Class Diagram

The class diagram models the object-oriented structure of the system. WebAPI represents the FastAPI application: it exposes the upload, progress, preview, and report operations, and manages many Job objects — one per grading request — each holding the job's state, progress counters, accumulated previews, and result records. For every job, the WebAPI spawns a worker that owns one OMRProcessorFast instance, the heart of the system: it loads the template's bubble grids, and its process_image() method runs the full per-sheet pipeline. The processor is parameterized by OMRConfig (threshold value, bubble box dimensions, and the fill-ratio cut-off), delegates geometric correction to the Calibration module, scores against the AnswerKey loaded from the uploaded CSV, and emits one SheetRecord per sheet, which the Job accumulates for the final CSV export.

Object Diagram

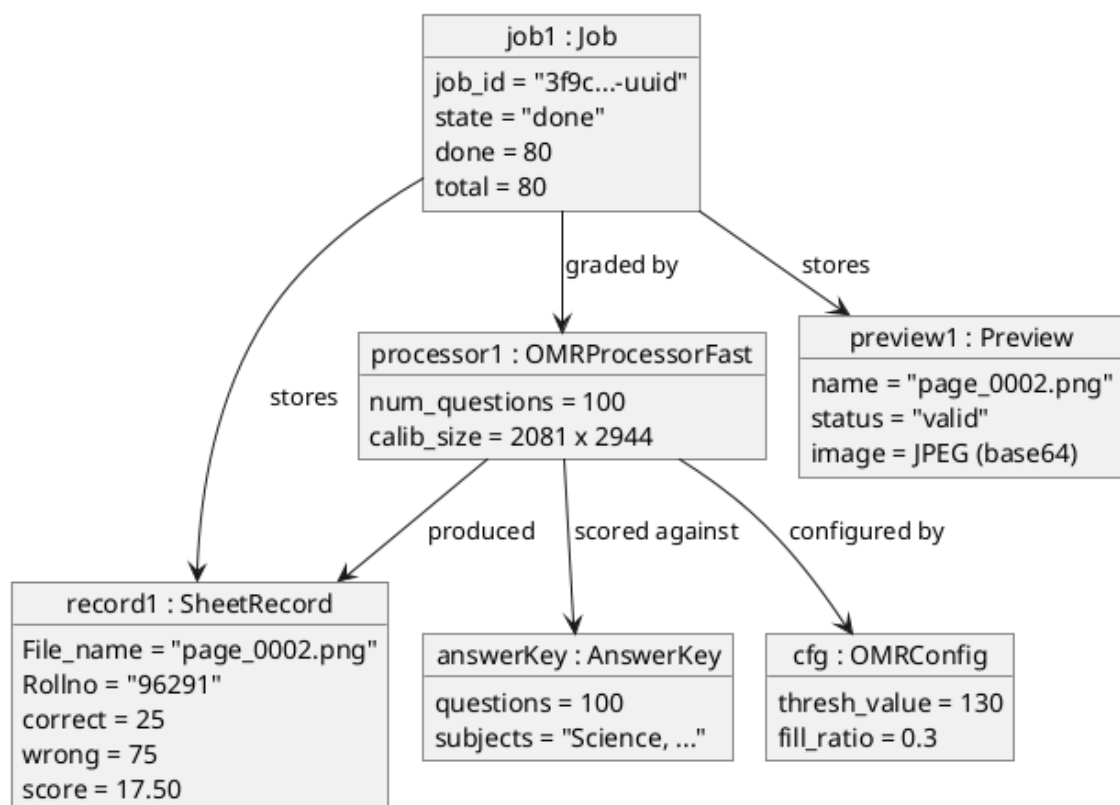


Figure 3.4: Object Diagram

The object diagram shows a runtime snapshot of a completed job. The job1 object has finished grading all 80 uploaded sheets. It was graded by processor1, an OMRProcessorFast configured for a 100-question template calibrated to a 2081×2944-pixel canonical sheet, using the cfg configuration (threshold 130, fill ratio 0.3). One of the produced records, record1, shows a decoded roll number 96291 with 25 correct and 75

wrong answers for a net score of 17.50 after negative marking; the corresponding preview1 holds the annotated JPEG that the examiner sees in the browser.

Dynamic Modeling using Sequence Diagram

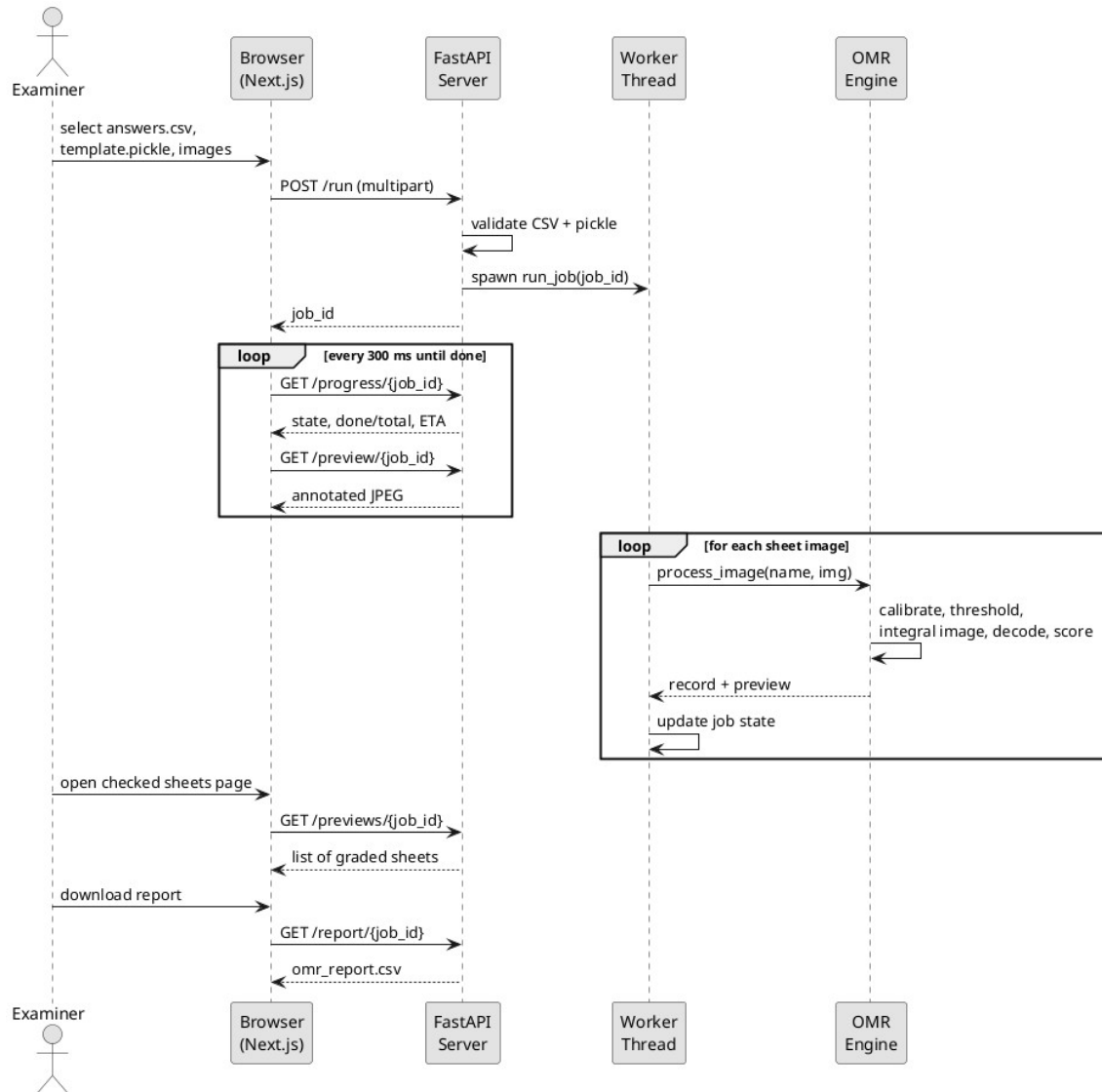


Figure 3.5: Sequence Diagram of the Grading Workflow

The sequence diagram illustrates the main scenario: an examiner submits a batch and receives graded results. The browser posts the three uploads to /run; the server validates the answer key and template, spawns the worker thread, and immediately returns a job ID. From then on two loops proceed concurrently: the worker grades one image at a time through the OMR engine (calibrate, threshold, integral image, decode, score) and updates the shared job state, while the browser polls /progress every 300 milliseconds and refreshes the annotated preview. When the job completes, the examiner can browse every checked sheet and download the consolidated CSV report.

Process Modeling using Activity Diagram

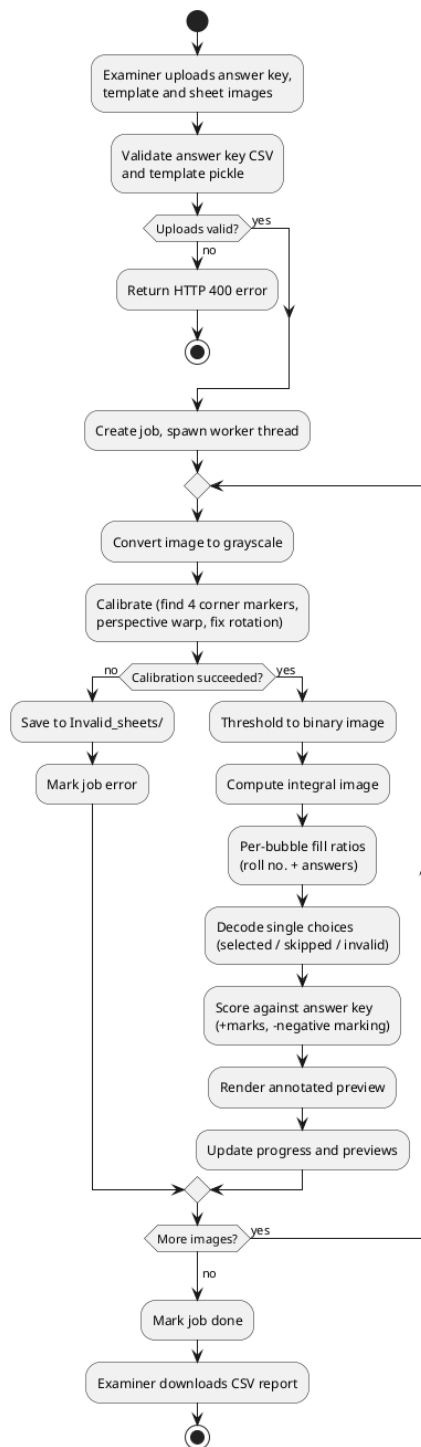


Figure 3.6: Activity Diagram of the Grading Pipeline

The activity diagram traces the control flow from upload to report. Invalid uploads are rejected before any grading starts. For each image in the batch, calibration either succeeds — leading to thresholding, integral-image fill-ratio measurement, decoding, scoring, and preview rendering — or fails, in which case the original photograph is saved to the

Invalid_sheets folder and the failure is reported. When every image has been processed the job is marked done and the examiner downloads the CSV report.

CHAPTER 4

SYSTEM DESIGN

4.1 Design

System design maps the analysis models onto a concrete architecture: the processing pipeline every sheet passes through, the modules that implement it, and the interfaces between the browser, the web service, and the vision engine.

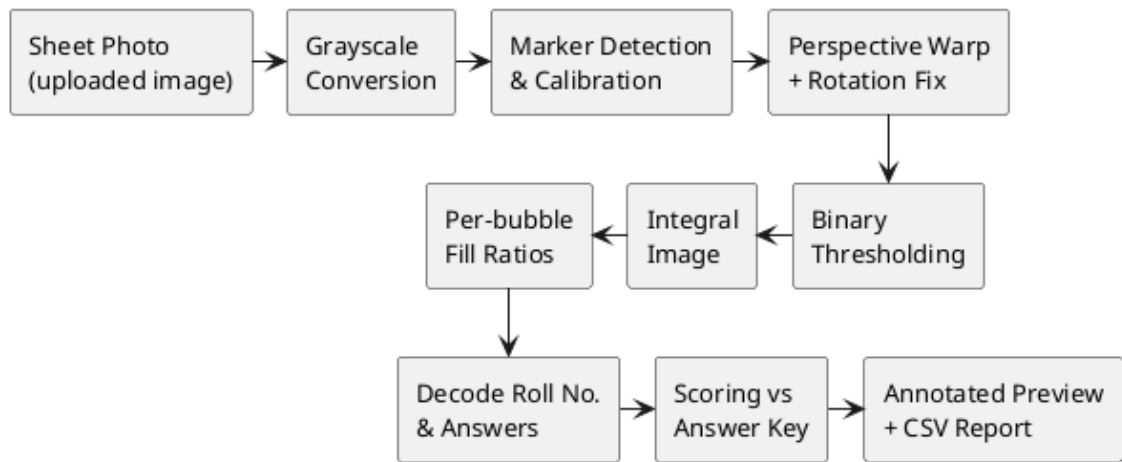


Figure 4.1: High-Level Design of the System

The high-level design in Figure 4.1 shows the per-sheet pipeline. Each uploaded photograph is converted to grayscale and passed to marker detection and calibration, which finds the four corner registration circles, orders them, and perspective-warps the sheet to the template's canonical size, fixing 180° rotation when detected. The corrected sheet is thresholded to a binary ink mask, over which a single integral image is computed. Fill ratios for all roll-number and answer bubbles are then evaluated in one vectorized pass, decoded into a roll number and per-question responses, scored against the answer key, and finally rendered as an annotated preview while the results accumulate into the CSV report.

4.1.1 Refinement of Class Diagram

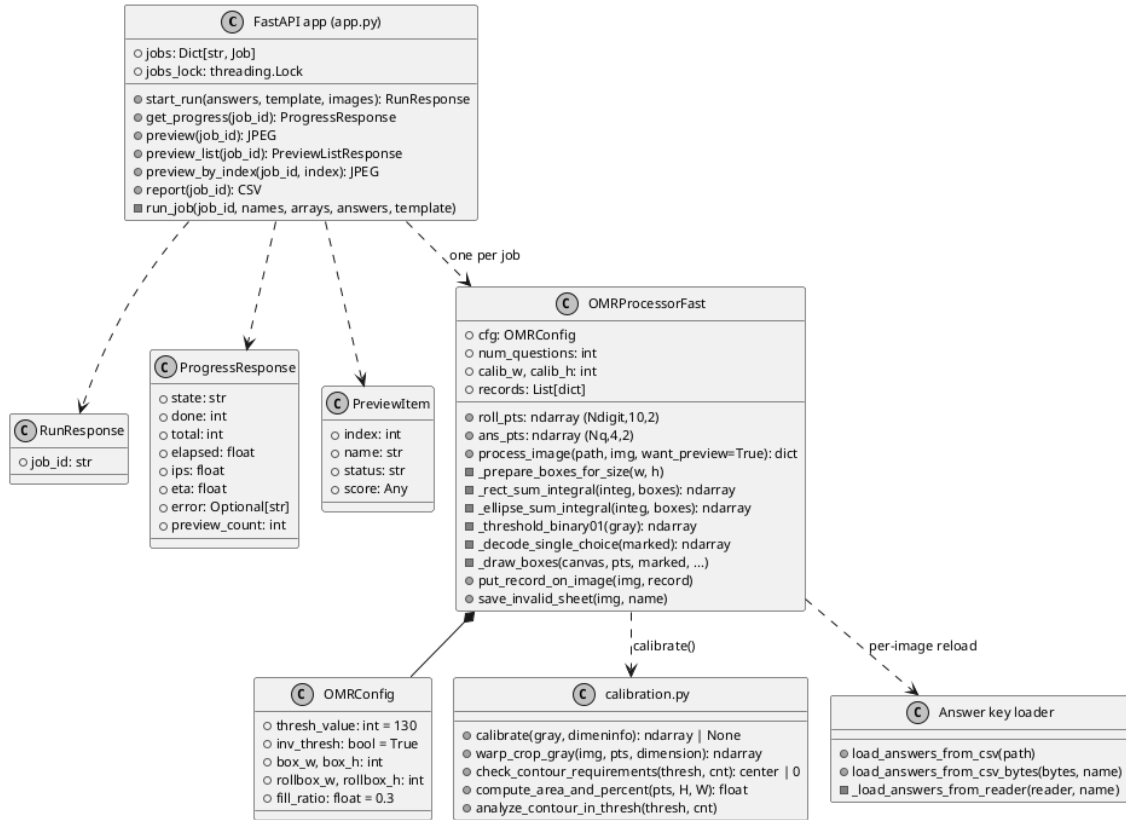


Figure 4.2: Refined Class Diagram

The refined class diagram adds the concrete attributes, types, and methods of the implementation. The FastAPI application holds the in-memory jobs dictionary guarded by a threading lock, and serializes its responses through the Pydantic models `RunResponse`, `ProgressResponse`, and `PreviewItem`, which give the API a typed, self-documenting contract. `OMRProcessorFast` exposes the template geometry as NumPy arrays — `roll_pts` with shape `(Ndigits, 10, 2)` and `ans_pts` with shape `(Nq, 4, 2)` — and its private methods realize each pipeline stage: `_threshold_binary01()` produces the ink mask, `_prepare_boxes_for_size()` precomputes every bubble's bounding box, `_ellipse_sum_integral()` measures ink inside each inscribed ellipse via the integral image, and `_decode_single_choice()` converts per-bubble booleans into selected/skipped/invalid codes. The calibration module and the answer-key loader remain separate, dependency-free units, which kept them independently testable during development.

4.1.2 Refinement of Sequence Diagram

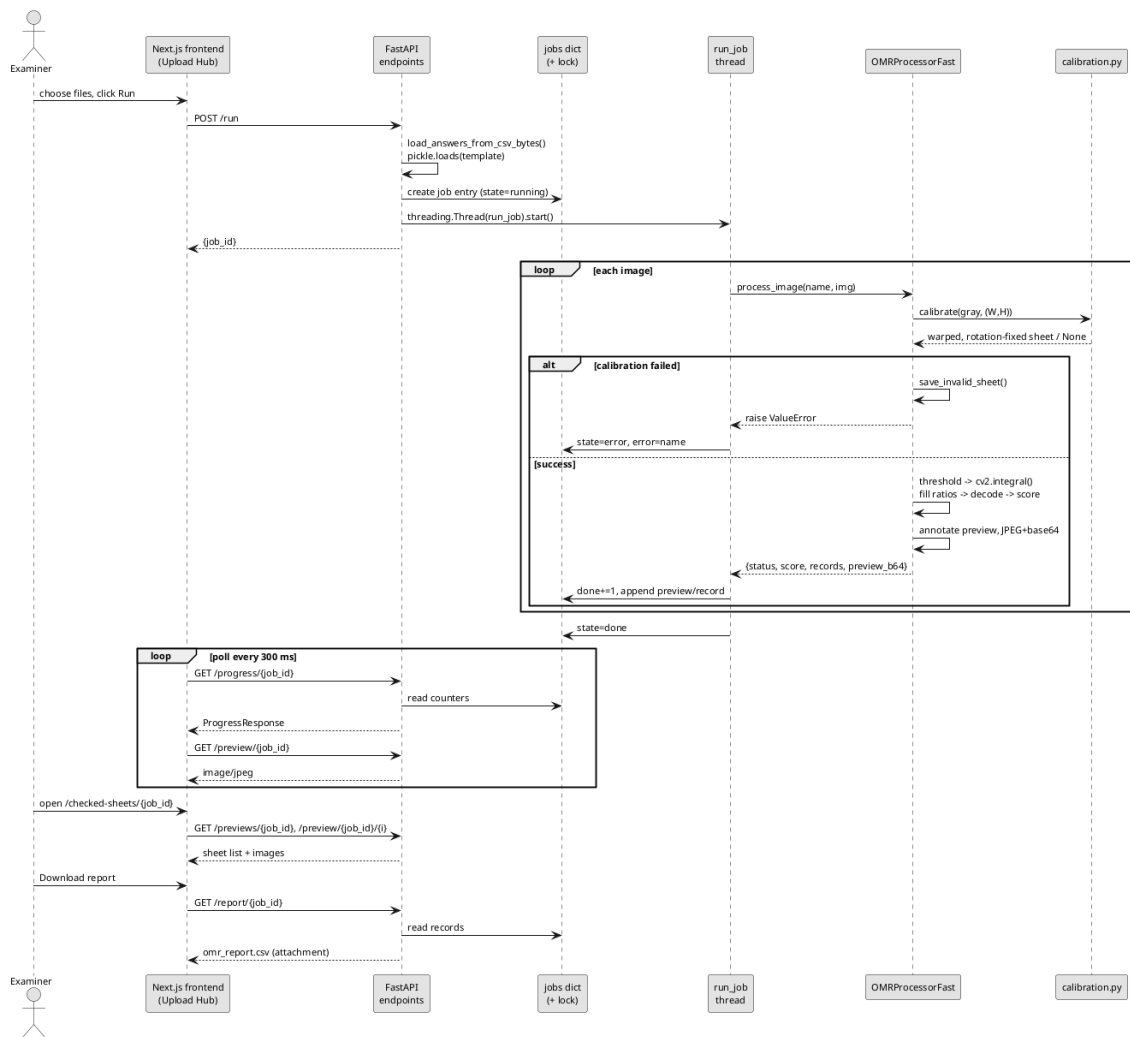


Figure 4.3: Refined Sequence Diagram

The refined sequence diagram names the concrete endpoints and internal calls. POST /run validates uploads with `load_answers_from_csv_bytes()` and `pickle.loads()` before creating the job entry under the lock and starting the daemon thread. Inside the worker loop, `process_image()` first calls `calibrate()`; on failure the engine saves the sheet to `Invalid_sheets` and the worker records the failing file name in the job's error state, while on success the engine thresholds, integrates, decodes, scores, and returns the record together with a base64-encoded JPEG preview. Meanwhile the browser polls GET /progress/{job_id} and refreshes GET /preview/{job_id}; the review page uses GET /previews/{job_id} for the sheet list and GET /preview/{job_id}/{index} for individual images, and GET /report/{job_id} streams the final CSV as a file download.

4.1.3 Refinement of Activity Diagram

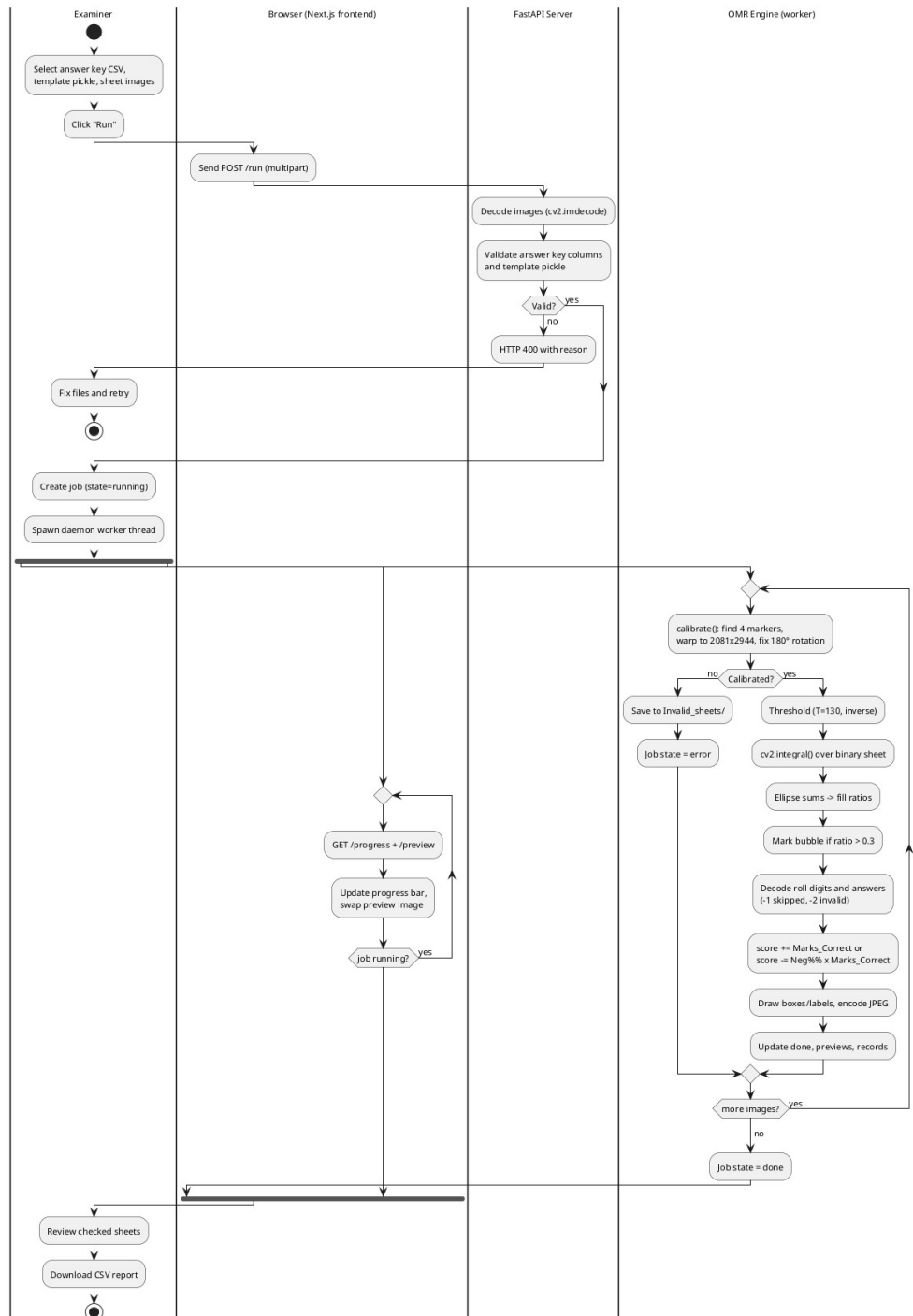


Figure 4.4: Refined Activity Diagram

The refined activity diagram partitions the flow into swimlanes for the examiner, the browser script, the FastAPI server, and the OMR worker, and annotates each action with its concrete parameters: the canonical warp size (2081×2944), the binarization threshold (130, inverse), the fill-ratio cut-off (0.3), and the decoding codes (−1 skipped, −2 invalid). The fork after job creation makes the system's concurrency explicit: grading and

progress-polling proceed in parallel until the worker marks the job done, after which the examiner reviews the checked sheets and downloads the report.

4.1.4 Component Diagram

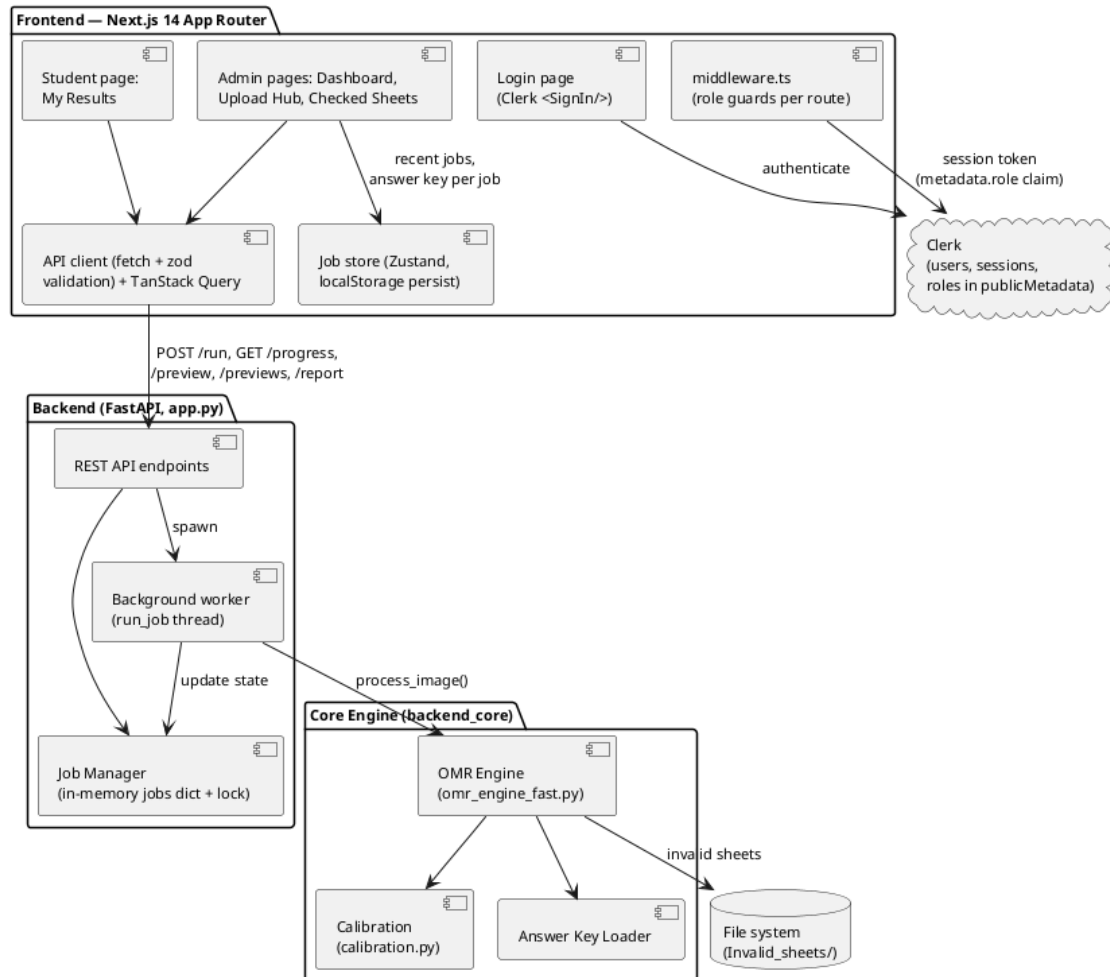


Figure 4.5: Component Diagram

The component diagram shows the system's modular organization. The frontend is a Next.js 14 App Router application: the login page embeds Clerk's sign-in component, server-side middleware guards every route using the role carried in the Clerk session token, and the role-specific pages — the administrator's Dashboard, Upload Hub, and Checked Sheets, and the student's My Results — all reach the backend through a single API client that validates every response with zod schemas and polls through TanStack Query. A small Zustand store persisted to localStorage remembers recent jobs and each job's answer key, which powers the client-side analytics. The FastAPI backend delegates to the job manager (the locked in-memory jobs dictionary) and the background worker, which drives the core engine package: the OMR engine performs detection, decoding, and

scoring; the calibration component performs geometric correction; the answer-key loader parses and validates the marking scheme; and failed calibrations are written to the file system.

4.1.5 Deployment Diagram

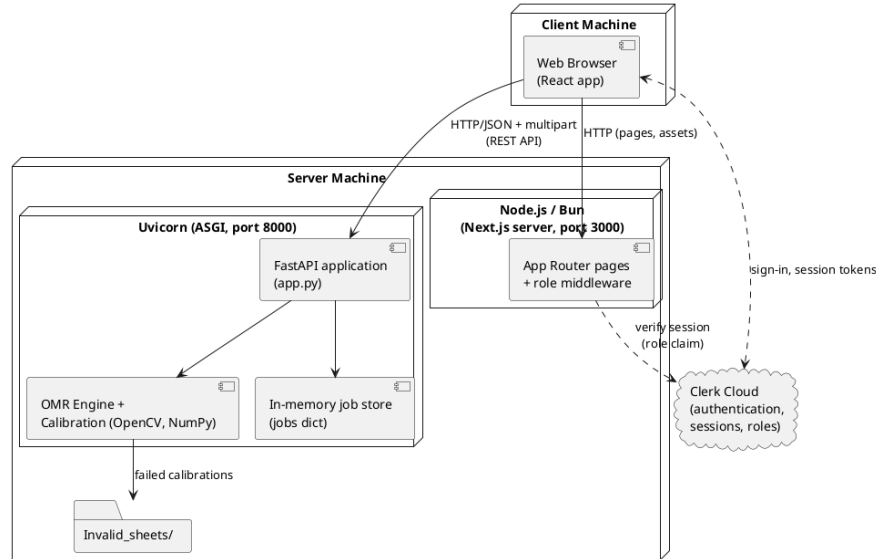


Figure 4.6: Deployment Diagram

The deployment diagram shows the physical arrangement. The browser loads the React application from the Next.js server (port 3000) and authenticates against the Clerk cloud; the resulting session token carries the user's role claim, which the Next.js middleware verifies on every request. Grading traffic then flows directly between the browser and the FastAPI backend on port 8000 — multipart uploads, JSON progress, JPEG previews, and CSV reports — where Uvicorn hosts the OMR engine with its OpenCV/NumPy dependencies and the in-memory job store. The only persistent artifact is the `Invalid_sheets/` folder holding photographs that failed calibration; for a single-institute deployment both server nodes can share one physical machine.

4.2 Algorithm Details

The system implements its pipeline with four classical computer-vision algorithms, described below in processing order.

a. Marker Detection and Perspective Calibration

The calibration algorithm locates the four solid circular registration marks printed near the sheet corners and uses them to undo the photograph's geometric distortion:

- Threshold the grayscale photograph (inverse binary, $T = 180$) so dark ink becomes foreground, and extract external contours using border following [3].
- Filter contours by area: a candidate marker must occupy between 0.05% and 0.1% of the page area, computed as contour area divided by image area.
- Verify circularity: the candidate's minimum-area rectangle must be nearly square (aspect ratio check), and the dark pixel count inside the contour must agree with the area πr^2 of the circle inscribed in that rectangle to within 20%.
- Collect four distinct markers by scanning the contour list from both ends in both contour order and vertical order, then sort their centres into top-left, top-right, bottom-right, bottom-left using the sum and difference of their coordinates.
- Estimate the perspective transform mapping the four centres to the corners of the template's canonical rectangle and warp the full image to the template size (2081×2944 pixels for the sheet used in testing).
- Resolve 180° ambiguity: compare the ink density in small windows at the midpoint of the top marker pair versus the bottom pair; if the sheet's known asymmetry appears inverted, rotate the warped image by 180°.

If fewer than four valid markers are found, calibration returns failure and the sheet is saved for manual review.

b. Binarization and Integral Image

The calibrated grayscale sheet is binarized with a fixed inverse threshold ($T = 130$): pixels darker than T become 1 (ink) and all others 0. Over this 0/1 mask the integral image ii is computed, defined so that each entry holds the sum of all mask values above and to the left:

$$ii(x, y) = \sum b(x', y') \text{ for all } x' \leq x, y' \leq y$$

The ink sum of any rectangle $(x1, y1, x2, y2)$ then costs only four lookups regardless of its size [2]:

$$S = ii(x2, y2) - ii(x1, y2) - ii(x2, y1) + ii(x1, y1)$$

c. Ellipse Fill-Ratio Measurement and Decoding

For every bubble centre recorded in the template, a bounding box of the bubble's average printed size is placed around the centre. Because bubbles are elliptical, ink is summed over the ellipse inscribed in the box rather than the whole box: for each pixel row of the

box, the ellipse equation gives the horizontal span inside the ellipse, and that one-row strip is summed with a four-lookup rectangle query, so the ellipse sum costs one query per row. The fill ratio is this sum divided by the ellipse area:

$$fill = S_{ellipse} / (\pi \cdot a \cdot b), \text{ marked} \Leftrightarrow fill > 0.3$$

where a and b are the ellipse semi-axes. Each question's four booleans (and each roll digit's ten booleans) are decoded by counting marks: exactly one mark selects that option; zero marks decode as skipped (-1); more than one decodes as invalid (-2). A roll number containing any skipped or invalid digit is reported as INV.

d. Scoring

Each decoded response is compared with the answer key row (Question, Subject, Marks_Correct, Negative_Percent, Answer). The score update is:

$$\begin{aligned} score &+= Marks_Correct, \text{ if } response = Answer \\ score &-= Negative_Percent \times Marks_Correct, \text{ if } response \neq Answer \end{aligned}$$

Skipped and invalid responses contribute nothing but are counted separately. The same update is applied to a per-subject accumulator keyed by the question's Subject, producing the subject-wise marks_<Subject> columns of the final report.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

Backend Tools

The backend is written in Python using the FastAPI web framework served by Uvicorn. FastAPI was chosen for its typed request/response models (Pydantic), automatic API documentation, and native multipart upload handling. All image processing uses OpenCV (cv2) with NumPy for vectorized array computation: OpenCV supplies image decoding, grayscale conversion, thresholding, contour detection, perspective warping, integral images, drawing primitives, and JPEG encoding, while NumPy implements the batched geometry and boolean decoding over all bubbles at once. Job concurrency uses the Python standard library's threading module, and the answer key is parsed with the standard csv module. No database is used: job state is held in memory for the lifetime of the server process.

Frontend Tools

The frontend is a Next.js 14 (App Router) application written in TypeScript with React 18. Authentication is provided by Clerk: users sign in through Clerk's hosted components, each user's role lives in their Clerk profile (publicMetadata), and server-side middleware reads the role from the session token to guard every route. Styling uses Tailwind CSS with Radix-based (shadcn/ui-style) components; charts are drawn with Recharts and toast notifications with Sonner. Server state is managed by TanStack Query, which polls job progress every 300 milliseconds while a job runs; client state — the active job, recent jobs, the pass threshold, and each job's parsed answer key — lives in a Zustand store persisted to localStorage; and every backend response and uploaded CSV is runtime-validated with Zod schemas, from which the TypeScript types are inferred. Bun serves as the package manager and development runtime.

Development Tools

Version control uses Git with GitHub hosting. Code quality is enforced with pre-commit hooks managed by lefthook, running oxlint for JavaScript linting and prettier for formatting; Python code is linted with ruff. Development was done in Visual Studio Code on Linux.

5.1.2 Implementation Details of Modules

Web API and Job Management Module (app.py)

This module exposes the REST interface listed in Table 5.1 and owns job lifecycle management. POST /run reads the uploads, decodes every image with cv2.imdecode (silently skipping files that are not decodable images), validates the answer key and template eagerly so the user gets an immediate HTTP 400 with the exact reason on bad input, creates the job entry under a lock, and starts a daemon worker thread before returning the job ID. The worker constructs one OMRProcessorFast for the whole batch — so the template is parsed and bubble boxes prepared only once — then processes images in order, updating the shared job dictionary (progress counters, latest preview, per-sheet records) under the same lock the request handlers use to read it. The report endpoint builds the CSV entirely in memory from the accumulated records, deriving the column order (file name, roll number, Q1...QN, subject-wise marks, counters, score) from the record keys.

Table 5.1: REST API Endpoints of the System

Method	Endpoint	Purpose
GET	/	Upload page (answers, template, images)
POST	/run	Validate uploads, start grading job, return job_id
GET	/progress/{job_id}	Job state, done/total, rate, ETA (polled every 300 ms)
GET	/preview/{job_id}	Latest annotated sheet preview (JPEG)
GET	/previews/{job_id}	List of all graded sheets (name, status, score)
GET	/preview/{job_id}/{index}	Annotated preview of one specific sheet (JPEG)
GET	/checked-sheets/{job_id}	Review page for paging through graded sheets
GET	/report/{job_id}	Export all results as a CSV attachment

OMR Engine Module (backend_core/omr_engine_fast.py)

OMRProcessorFast implements the grading pipeline. Its constructor unpickles the template, reads the canonical sheet size and the average printed bubble dimensions (from which the detection box sizes are derived), transposes the roll-number grid to (Ndigits, 10) form, and validates the geometry — every roll row must have exactly ten points and every question exactly four. process_image() executes the per-sheet pipeline described in Chapter 4: calibration, inverse binary thresholding at $T = 130$, a single cv2.integral() over the ink mask, vectorized ellipse fill-ratio measurement for all roll and answer bubbles, single-choice decoding, and scoring with subject-wise accumulation using defaultdict counters. The same method renders the annotated preview: every bubble box is drawn, marked answers are labelled correct, incorrect, or invalid in colour, the roll number and score summary is overlaid in the top-right corner, and the canvas is downscaled and JPEG-encoded to base64 for transport. The answer-key loader validates the five required columns and rejects non-numeric or negative mark values with the exact line number of the offending row.

Calibration Module (calibration.py)

calibrate() implements marker detection as designed in Section 4.2: inverse thresholding at $T = 180$, external contour extraction, area-percentage filtering (0.05–0.1% of page area), and the squareness and inscribed-circle-area checks of check_contour_requirements(). Candidate scans run from both ends of the contour list in two orderings so the four corner marks are found regardless of contour enumeration order. The four centres are ordered by coordinate sums and differences, warp_crop_gray() estimates the perspective transform with cv2.getPerspectiveTransform and warps the sheet to the template size, and the ink-density comparison around the top and bottom marker midpoints triggers a 180° rotation when the sheet was captured upside down. On any failure the function returns None, which the engine treats as an invalid sheet.

Frontend Application Module (frontend/)

The Next.js application is organized around the App Router: /login hosts Clerk's sign-in; a shared dashboard shell provides the sidebar navigation and the header with the signed-in account; /admin, /admin/upload, and /admin/checked-sheets/{jobId} implement the administrator workspace; and /student implements the student portal. src/middleware.ts declares the route guards — /super-admin requires SUPER_ADMIN, /admin admits

SUPER_ADMIN and ORG_ADMIN, /student admits SUPER_ADMIN and STUDENT — redirecting unauthenticated visitors to the login page and role-less or under-privileged users to the root page. src/lib/api-client.ts wraps every backend endpoint and validates each response against zod schemas, and src/lib/analytics.ts computes all dashboard analytics (pass rate, score distribution, subject strength, hardest questions) in the browser from the report CSV, using the answer key the administrator uploaded, which the Zustand store retains per job.

5.1.3 Working of the System (System Walkthrough)

This section demonstrates the complete working of the delivered platform, captured live from the running system while grading the 80-sheet test batch described in Section 5.3. Access is role-based: every user signs in through Clerk, and what they can see and do is decided by the role stored on their account, summarized in Table 5.2.

Table 5.2: User Roles and Access in the Platform

Role	Accessible areas	Purpose and usefulness
SUPER_ADMIN	Everything: the admin workspace, the student portal, and the reserved /super-admin area	Platform oversight. Passes every route guard, so a super admin can run or inspect any grading batch, open any dashboard, and audit the system end to end; the reserved area is where organisation and user management will live.
ORG_ADMIN	/admin (analytics dashboard), /admin/upload (Upload Hub), /admin/checked-sheets/{jobId} (sheet review)	The examiner/institute role. Runs grading jobs, watches live progress, studies batch analytics (pass rate, distribution, subject strength, hardest questions), reviews every checked sheet, and exports the CSV report.
STUDENT	/student (My Results) only	Self-service result access. Looks up their own result with the exam's job id and their roll number; sees their score, rank, batch comparison, and their own annotated sheet — and nothing belonging to other students.

Step 1 — Signing in and role routing.

Every user starts at the login page (Figure 5.1), which embeds Clerk's hosted sign-in component; Clerk handles the credentials, new-device verification, and session issuance. Each user's role is stored in their Clerk profile (publicMetadata.role) and carried into the session token, and the application's server-side middleware checks it on every request:

/admin routes admit SUPER_ADMIN and ORG_ADMIN, /student admits SUPER_ADMIN and STUDENT, and /super-admin admits only SUPER_ADMIN. After sign-in the user lands on the root page, which forwards them to their role's home — administrators to the dashboard, students to My Results — so a student typing an admin URL by hand is simply redirected away, and a signed-in user with no role assigned is shown setup instructions instead of a dashboard.

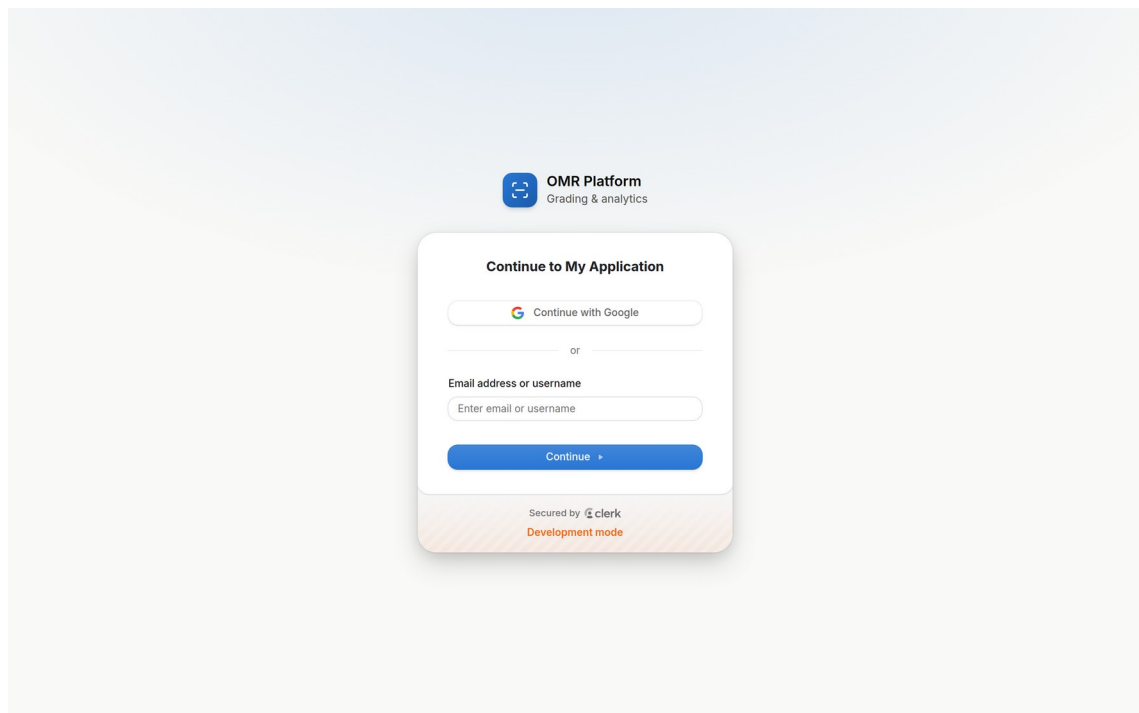


Figure 5.1: Login Page (Clerk Authentication)

Step 2 — The administrator's workspace.

An ORG_ADMIN signs in and lands on the Institute Dashboard. The sidebar offers Dashboard and Upload Hub (a Checked Sheets entry appears once a job is active), and the header shows the signed-in account. With no batch graded yet, the dashboard shows an empty state (Figure 5.2) offering two ways in: start a new grading job, or load an existing one by pasting its job id — useful when a colleague shares a batch or after reopening the browser.

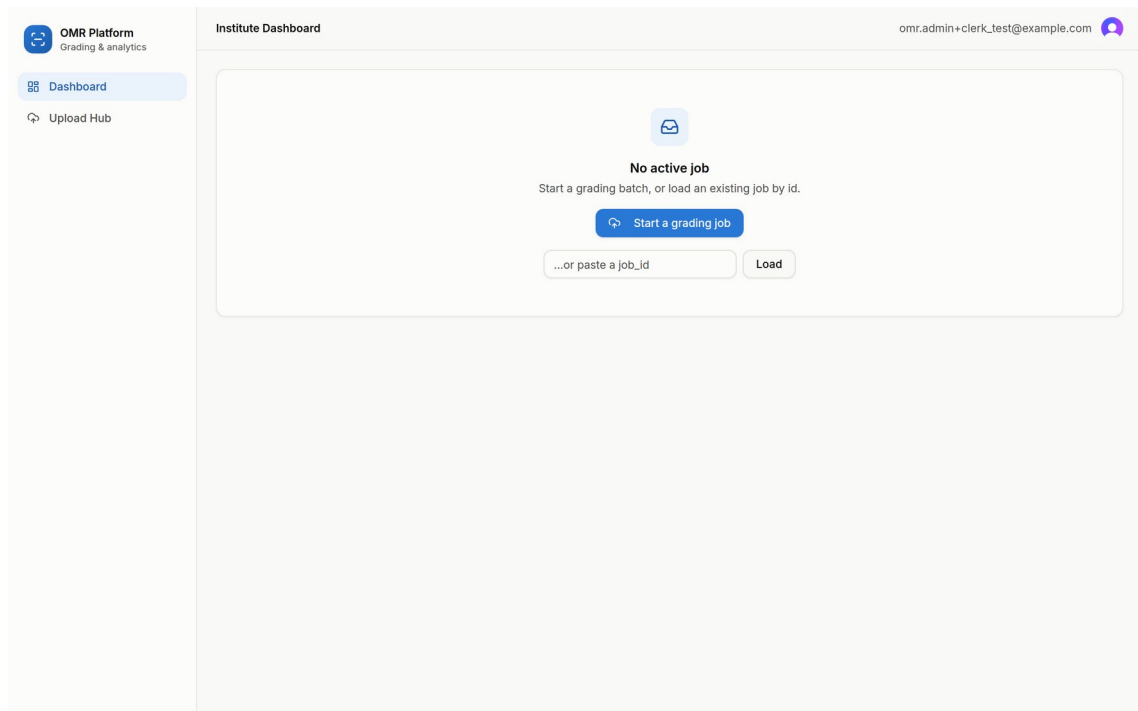


Figure 5.2: Admin Dashboard Before Any Job (Empty State)

Step 3 — Upload Hub: starting a grading job.

On the Upload Hub the administrator drags and drops the sheet photographs (each appears as a removable chip — here all 80 images of the test batch), then picks the answer key CSV and the template pickle (Figure 5.3). Before anything is sent, the answer key is parsed and validated in the browser with zod schemas, so a malformed key fails instantly with the offending row number; the validated key is also kept locally because it later powers the per-question analytics. Clicking Start grading posts everything to POST /run and the backend returns the job id.

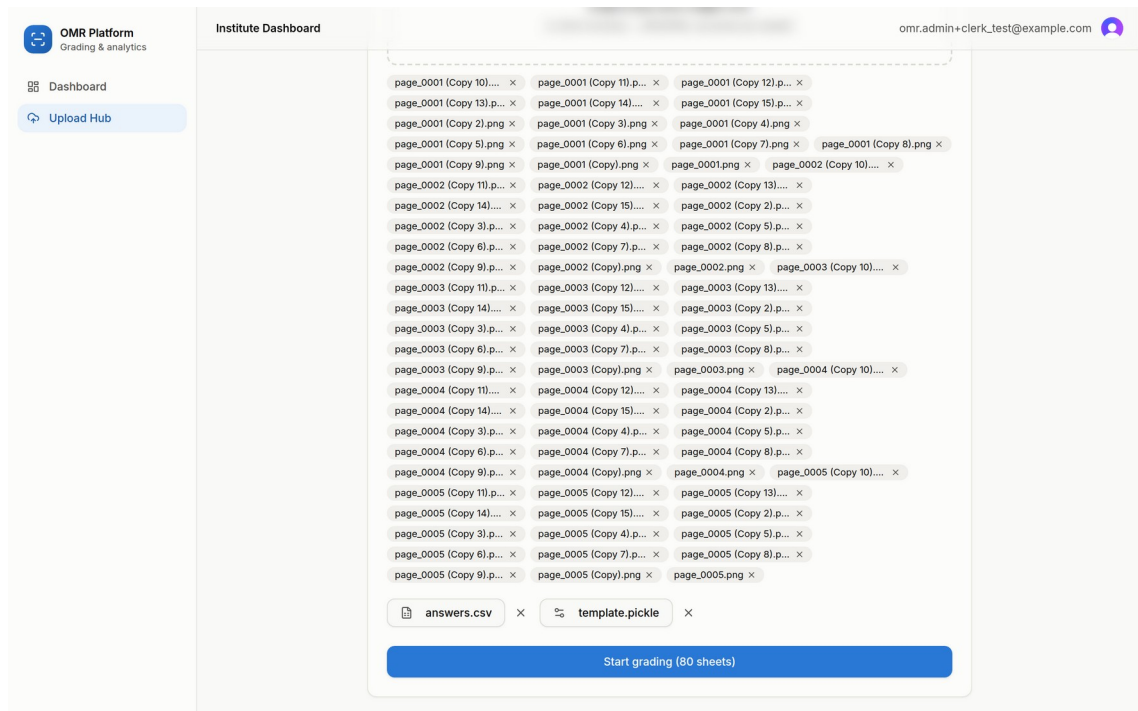


Figure 5.3: Upload Hub with Sheets, Answer Key, and Template Selected

Step 4 — Live grading progress.

While the backend worker grades the batch, the Upload Hub polls GET /progress every 300 milliseconds and renders a live job card (Figure 5.4): the progress bar, the processed count, the grading speed in sheets per second, the estimated time remaining, the file currently being graded, and the annotated preview of the most recently graded sheet, re-fetched each time the processed count advances. When the job completes, two actions appear: View analytics and Review sheets. If a sheet cannot be calibrated the card shows the failing file name and notes that the photograph was preserved server-side in Invalid_sheets/.

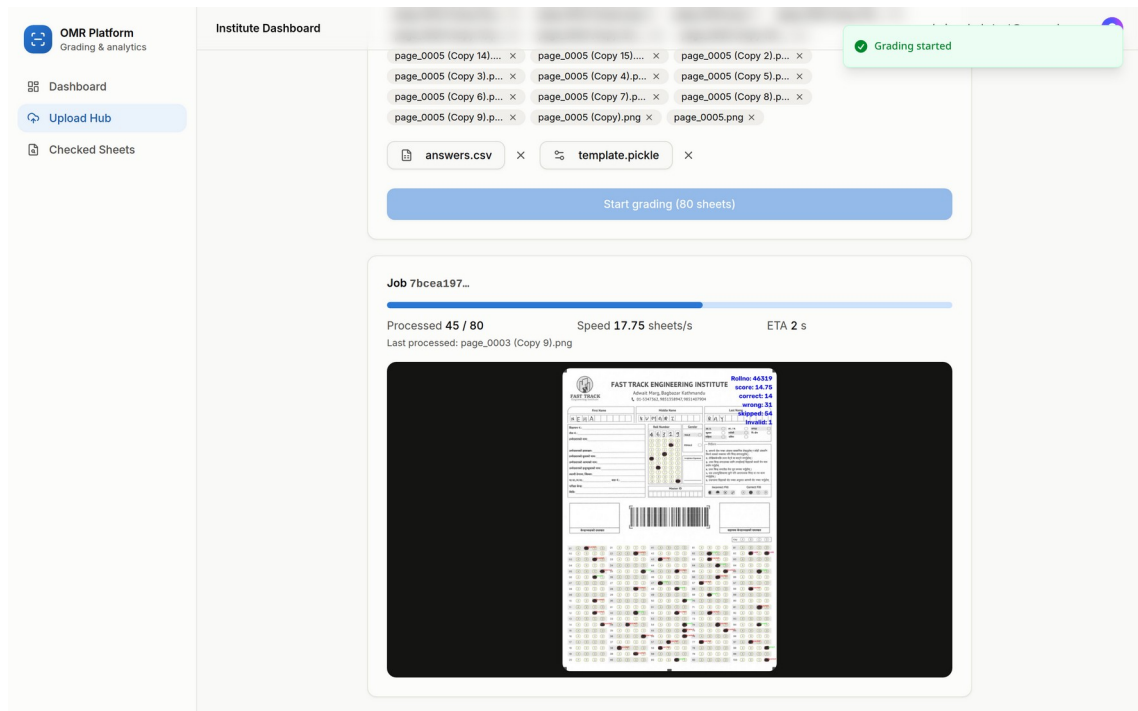


Figure 5.4: Live Grading in Progress (Job Card with Annotated Preview)

Step 5 — Exam analytics dashboard.

The Dashboard (Figure 5.5) turns the graded batch into decision-ready analytics, computed client-side from the job's report. Four KPI cards summarize the batch: students graded, passing rate (against a pass threshold of 40% of maximum marks, drawn as a progress ring), class average with a trend against the 50% benchmark, and the invalid-sheet count, which links straight to the review page. Three charts follow: the score distribution across percentage brackets, subject strength as average marks per subject, and the hardest questions ranked by incorrect-answer rate — telling the examiner exactly what to re-teach. Below, the student table lists every sheet with roll number, score, percentage, and pass/fail status; it is searchable by roll number or file name, filterable by pass/fail and a percentage range, and each row's View sheet button deep-links to that sheet in the review page. A job picker in the header switches between recent batches, and Export CSV downloads the full report.

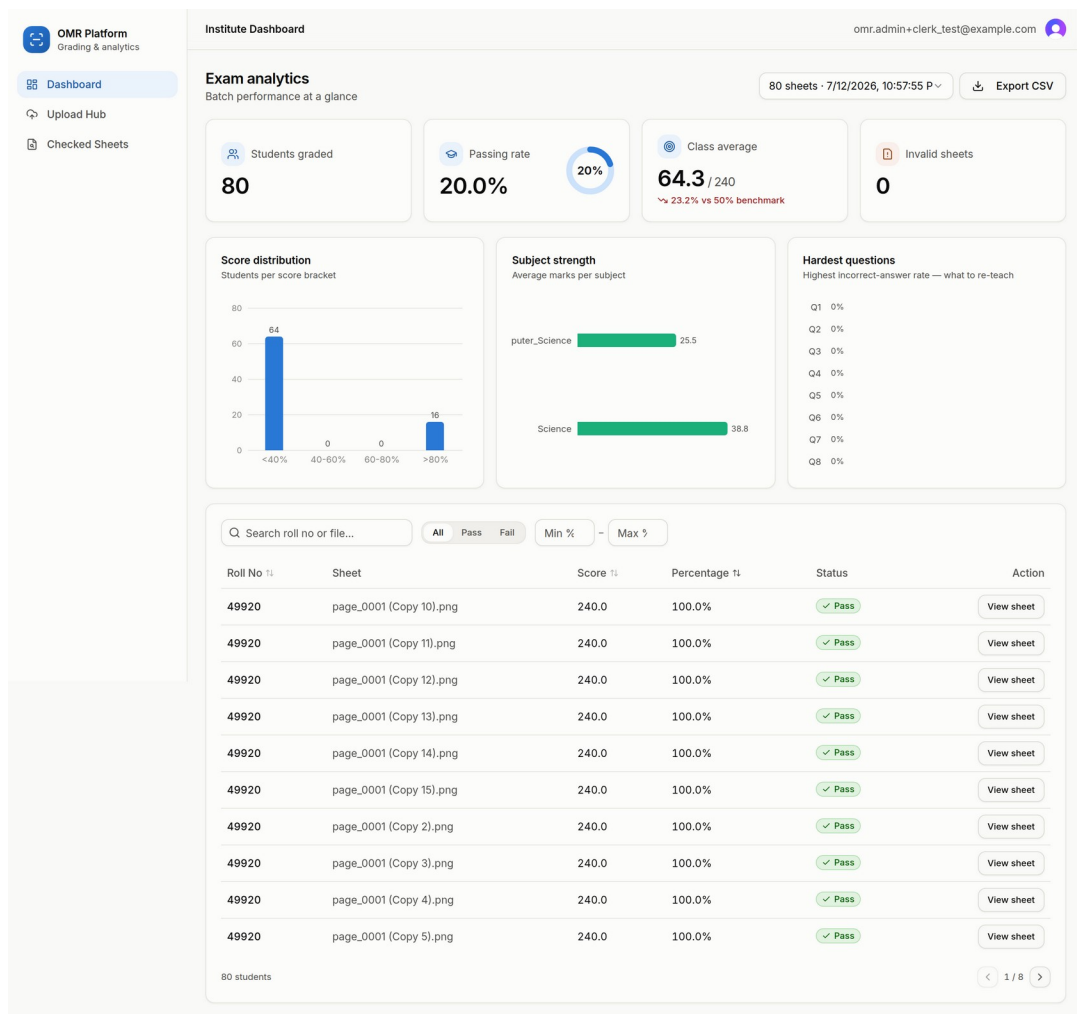


Figure 5.5: Exam Analytics Dashboard (KPIs, Charts, and Student Table)

Step 6 — Checked-sheets review.

The Checked Sheets page (Figure 5.6) is the auditing surface: a split-pane view with every graded sheet listed on the left (file name and score) and, on the right, the selected sheet's annotated image exactly as the engine graded it, its summary line (roll number, score, correct/wrong/skipped), and a paginated per-question table showing the student's response, the correct answer from the stored answer key, and a status badge (correct, wrong, skipped, or invalid). Any disputed mark can be resolved here by looking at the actual sheet.

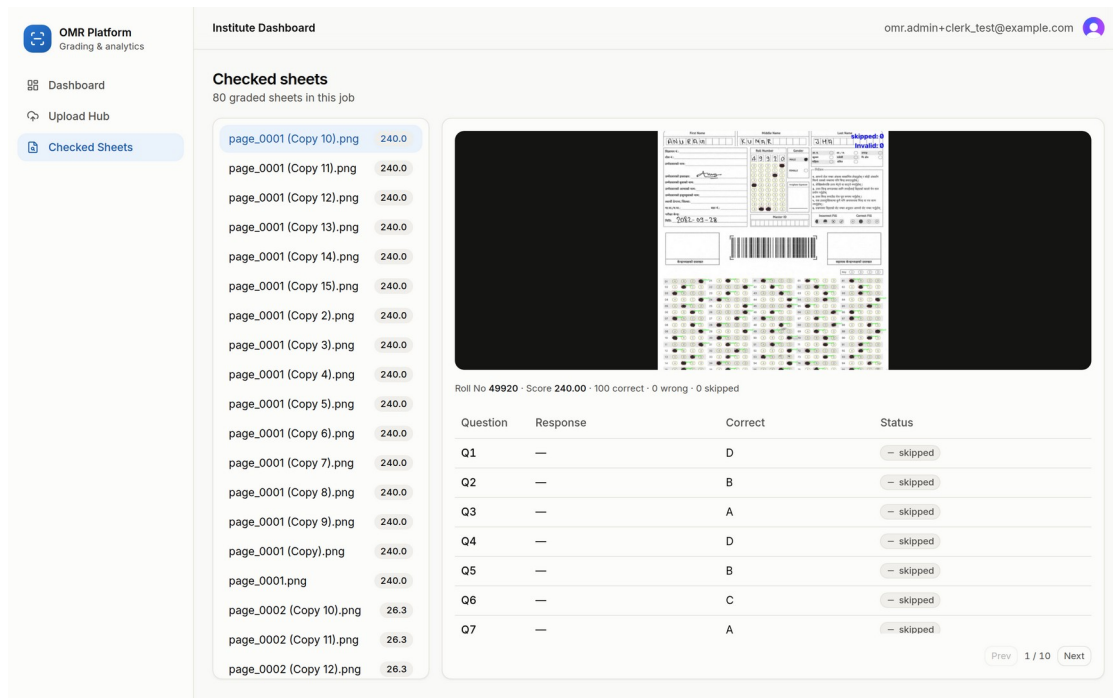


Figure 5.6: Checked-Sheets Review (Split-Pane with Per-Question Breakdown)

Step 7 — The student's portal.

A STUDENT signing in sees only My Results (Figure 5.7). The student enters the exam's job id (shared by the teacher) and their roll number — prefilled from their Clerk profile when the administrator has set it — and looks up their result. The scorecard shows four personal metrics: their score out of the maximum, the batch average with a you-versus-batch comparison, their rank in the batch (here 65 of 80), and their correct·wrong·skipped counts. Below, they see their own annotated sheet with the per-question breakdown — the same evidence the examiner sees, but only for their own sheet: the lookup matches a single roll number, and wrong job ids or roll numbers get a clear error instead of data. This gives students transparent, self-service access to results without exposing anyone else's.

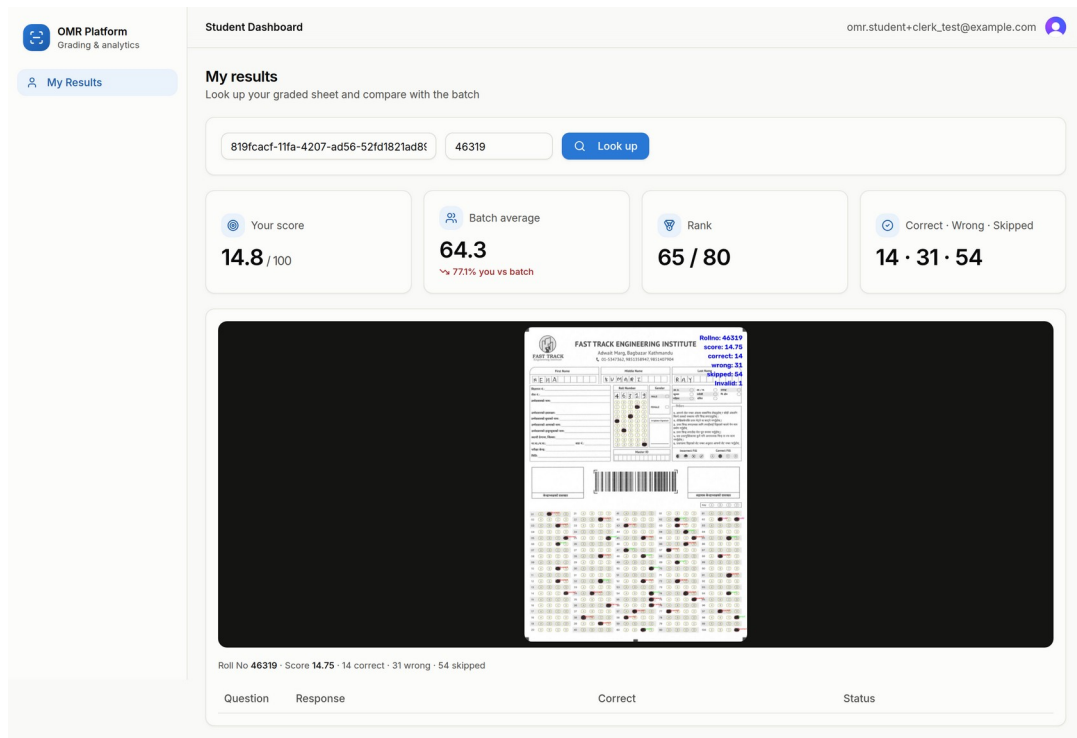


Figure 5.7: Student Portal — Personal Scorecard and Own Sheet Review

Step 8 — The super administrator.

A `SUPER_ADMIN` passes every route guard, so the same account can open the admin workspace and the student portal alike. Figure 5.8 shows the super admin account inspecting the batch by pasting its job id into the dashboard's load form — no matter which browser started the job, any batch alive on the server can be pulled up and audited. The guarded `/super-admin` route is reserved for the platform-management area (organisations, users, system settings), pending the corresponding backend endpoints.

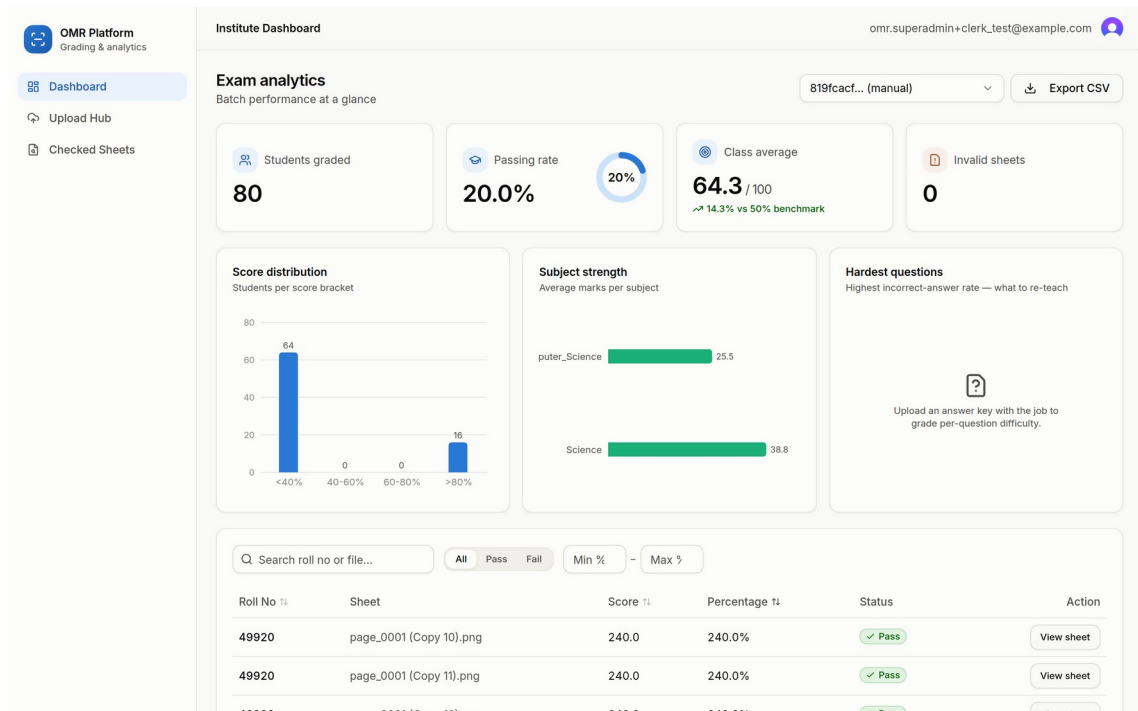


Figure 5.8: Super Admin Viewing a Batch Loaded by Job ID

Frontend workflow in detail.

Under the hood, TanStack Query drives all server state: the progress query re-fetches every 300 milliseconds while a job is running and stops automatically once it settles, and the report query runs only when a job reaches its final state. The live preview image is keyed by the processed count, so the browser fetches a new JPEG exactly when a new sheet has been graded and never in between. A Zustand store persisted to localStorage remembers up to twenty recent jobs, the active job, the pass threshold, and each job's parsed answer key — the key never comes back from the backend, so keeping the admin's uploaded copy is what enables hardest-question analytics and the per-question correct-answer column. All analytics are computed in the browser from the report CSV (there is no analytics endpoint), which keeps the backend unchanged and is fast at batch scale. Every backend response and every uploaded CSV passes through zod schema validation, so malformed data fails loudly with a precise message rather than corrupting the UI. Because the backend stores jobs only in process memory, a remembered job can outlive its server-side state; the dashboard detects this (progress returns unknown) and shows a clear warning to re-run the batch instead of failing silently.

5.1.4 Backend API Documentation (Swagger UI)

Because every endpoint is declared with typed parameters and Pydantic response models (RunResponse, ProgressResponse, PreviewItem, PreviewListResponse), FastAPI automatically generates an OpenAPI 3.1 specification for the whole backend, served as interactive Swagger documentation at /docs (Figure 5.9) and as machine-readable JSON at /openapi.json. The documentation groups the endpoints by tag — Pages, Jobs, Previews, and Reports — states each one's purpose, request schema, and response schema, and lets the API be exercised directly from the browser, which was also used during development to test endpoints without the frontend. This documentation stays correct by construction: it is derived from the same type declarations the server enforces at runtime.

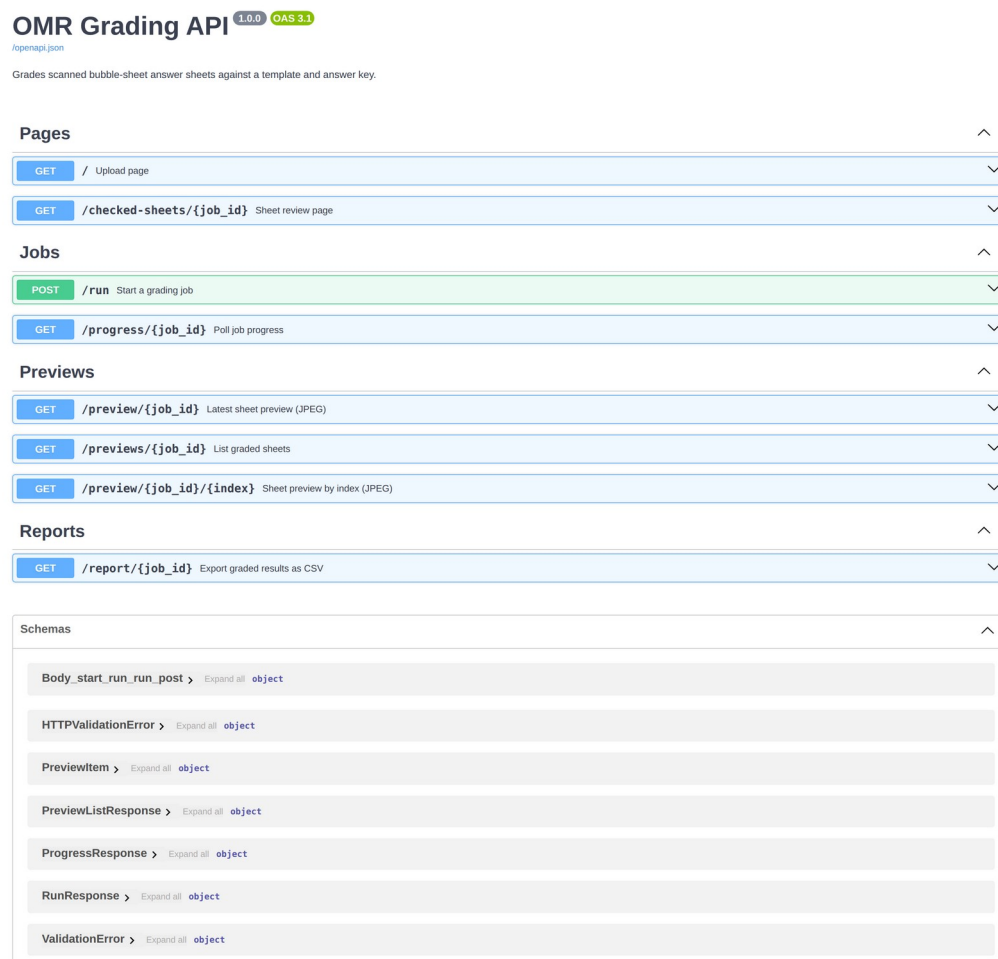


Figure 5.9: Auto-Generated API Documentation (Swagger UI, /docs)

5.2 Testing

Testing verified the system at two levels: unit tests exercising individual components with controlled inputs, and system tests exercising the complete workflow through the web interface with real photographed sheets.

5.2.1 Test Cases for Unit Testing

Table 5.3: Test Cases for Answer Key and Upload Validation

S.N.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
1	TC-UV-1	Reject answer key with a missing required column	CSV without the 'Answer' column	Error naming the missing column; HTTP 400	"Missing columns: ['Answer']" returned, job not started	Pass
2	TC-UV-2	Reject non-numeric marks value	Marks_Correct = "abc" on line 7	Error with the offending line number	"Invalid Marks_Correct 'abc' at line 7"	Pass
3	TC-UV-3	Reject negative negative-marking value	Negative_Percent = -0.25	Error: value must be ≥ 0	Error raised naming the question	Pass
4	TC-UV-4	Tolerate blank rows in the answer key	CSV with empty rows between questions	Blank rows skipped, remaining questions loaded in order	100 questions loaded, sorted by number	Pass
5	TC-UV-5	Reject empty or corrupt template upload	Zero-byte file / random bytes as template	HTTP 400 "Template file is empty / Invalid template file"	HTTP 400 with reason	Pass

Table 5.4: Test Cases for Bubble Detection and Decoding

S.N.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
1	TC-BD-1	Detect a clearly filled bubble	Question with option C filled (fill ratio ≈ 0.6)	Decoded as "C"	Decoded as "C", scored against key	Pass
2	TC-BD-2	Handle an unanswered question	Question with no bubble filled	Decoded as skipped; blank in report; no marks	Counted in "skipped", blank cell in CSV	Pass

				change		
3	TC-BD-3	Handle multiple marks on one question	Two options filled on one question	Decoded as invalid (INV); no marks change	“INV” in report, counted in “Invalid”	Pass
4	TC-BD-4	Handle ambiguous roll-number grid	One roll digit column with two filled bubbles	Roll number reported as “INV”	Rollno = “INV” in record and preview	Pass
5	TC-BD-5	Apply negative marking	Wrong answer on a 4-mark question with 25% negative	Score decreases by 1.0	Score -1.0; subject total -1.0	Pass

5.2.2 Test Cases for System Testing

Table 5.5: Test Cases for System Testing

S.N.	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Remarks
1	TC-ST-1	Grade a full batch end-to-end	80 photographed sheets, 100-question template, answer key	All sheets graded; progress reaches 80/80; state “done”	80/80 graded, previews and records complete	Pass
2	TC-ST-2	Correct an upside-down capture	Sheet photographed rotated 180°	Rotation detected and corrected; sheet graded normally	Preview upright, roll number and score correct	Pass
3	TC-ST-3	Reject an uncalibratable image	Photograph without corner markers	Image saved to Invalid_sheets/; job error names the file	File in Invalid_sheets/, error shown with file name	Pass
4	TC-ST-4	Export the CSV report	Completed job	CSV with Rollno, Q1–Q100, subject marks, counters, score per sheet	omr_report.csv downloaded with all columns	Pass
5	TC-ST-5	Poll an unknown job	Random job_id in /progress	State “unknown”, no crash	“unknown” returned	Pass

5.3 Result Analysis

Result analysis evaluated the complete system on a batch of 80 real photographed answer sheets, each containing 100 four-option questions and a five-digit roll-number grid, using the marking scheme of the accompanying answer key (4 marks per correct answer, 25% negative marking). The measurements below were taken on an ordinary development laptop with no GPU acceleration.

Table 5.6: Measured Performance on the Test Batch

Metric	Measured Value
Sheets in batch	80
Questions per sheet	100 (400 answer bubbles + 50 roll bubbles)
Sheets successfully calibrated	80 of 80 (100%)
Roll numbers decoded	80 of 80
Total processing time	12.3 seconds
Throughput	≈ 6.5 sheets per second
Average time per sheet	≈ 153 milliseconds (including preview rendering)
Score range observed	14.75 to 240.00 (with negative marking)

Calibration proved fully reliable on the test batch: all four registration marks were found in every photograph, including rotated captures, so no sheet was lost to geometric failure. Grading throughput of roughly 6.5 sheets per second means a 1,000-sheet examination can be graded in under three minutes — the practical bottleneck becomes photographing the sheets, not grading them. Profiling showed that the vectorized integral-image design is what makes this possible: measuring all 450 bubble regions of a sheet costs a single integral-image construction plus a few array lookups per bubble, and a substantial share of the remaining per-sheet time is spent rendering the annotated preview JPEG rather than grading itself. Consistently, in the live web application the grading worker sustained approximately 17.7 sheets per second (visible in the job card of Figure 5.4), because POST /run decodes all uploaded images into memory before the worker starts; the end-to-end 6.5 sheets-per-second figure additionally includes reading and decoding every photograph from disk. Spot-checking the annotated previews against the physical sheets confirmed that decoded responses, invalid detections (multiple marks), and negative-marking arithmetic matched manual evaluation on the inspected sheets, and the exported CSV totals were consistent with the per-question decodes.

CHAPTER 6

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

This project set out to make automated grading of multiple-choice answer sheets accessible without dedicated OMR hardware, and the delivered system achieves that goal. OMR-Web grades ordinary photographed bubble sheets through a browser: marker-based perspective calibration makes the pipeline robust to the skew, scale, and rotation of real photographs; integral-image fill-ratio analysis makes bubble detection both accurate and fast; and a configurable answer key supports per-question marks, negative marking, and subject-wise totals. The web layer turns the pipeline into a usable tool: grading runs as a background job while the examiner watches live progress, every graded sheet is returned as an annotated preview in which each decision can be visually verified, failed sheets are preserved for manual review, and the complete result exports as a standard CSV. Access is role-aware end to end: Clerk authentication and server-side route guards give the organisation administrator the grading and analytics workflow — including a dashboard of passing rate, score distribution, subject strength, and hardest questions — give each student transparent, self-service access to exactly their own scorecard and sheet, and give the super administrator full oversight of the platform.

Testing on 80 real photographed sheets of 100 questions each demonstrated 100% calibration success and a throughput of roughly 6.5 sheets per second on an ordinary laptop, confirming that classical, interpretable computer vision — thresholding, contour analysis, homography, and integral images — is sufficient for practical, verifiable automated grading. Beyond the delivered software, the project provided direct experience with the full engineering cycle: analysing a real institutional problem, designing an image-processing pipeline, implementing it as a concurrent web service, and validating it against real-world data.

6.2 Future Recommendations

Although the system meets its objectives, several enhancements would extend its robustness and reach:

- **Adaptive thresholding:** replacing the fixed binarization threshold with adaptive or Otsu thresholding computed per sheet would improve robustness to uneven lighting and different paper or printer characteristics.
- **Integrated template builder:** the bubble-layout template is currently produced by an external tool; a built-in visual template designer would make the system self-contained for new sheet layouts.
- **Persistent job storage:** storing jobs and results in a database (e.g., SQLite) would preserve grading history across server restarts and let each organisation browse its own past batches instead of sharing job ids manually.
- **Super-admin management area:** the /super-admin route is already role-guarded; building it out with organisation, user, and system management requires the corresponding backend endpoints and would complete the platform's administration story.
- **PDF and mobile capture input:** accepting multi-page PDF scans and providing a phone-camera capture flow with on-device framing guides would streamline sheet intake.
- **Flexible sheet formats:** supporting variable option counts, multiple-answer questions, and partial-credit schemes would widen the range of examinations the system can grade.
- **Review and override workflow:** letting the examiner correct an individual decoded answer directly on the preview, with the score recomputed, would close the loop on the rare ambiguous mark.

References

- [1] E. M. de Elias, P. M. Tasinaffo, and R. Hirata Jr., “Optical Mark Recognition: Advances, Difficulties, and Limitations,” *SN Computer Science*, vol. 2, art. 367, 2021.
- [2] P. Viola and M. J. Jones, “Robust Real-Time Face Detection,” *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [3] S. Suzuki and K. Abe, “Topological Structural Analysis of Digitized Binary Images by Border Following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.
- [4] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [5] G. Bradski, “The OpenCV Library,” *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [7] U. Deshmukh, “OMRChecker: Grade exams fast and accurately using a scanner or your phone,” GitHub repository. [Online]. Available: <https://github.com/Udayraj123/OMRChecker>
- [8] S. Ramírez, “FastAPI,” [Online]. Available: <https://fastapi.tiangolo.com/>
- [9] N. Sattayakawee, “Test Scoring for Non-Optical Grid Answer Sheet Based on Projection Profile Method,” *International Journal of Information and Education Technology*, vol. 3, no. 2, pp. 273–277, 2013.

APPENDICES

Appendix A: Screenshots

FAST TRACK ENGINEERING INSTITUTE
 Adwait Marg, Bagbazar Kathmandu
 01-5347362, 9851358947, 9851407904

Registration Form:

First Name: H I M A N I
 Middle Name: K U M A R I
 Last Name: S A H

Roll Number: 5 5 8 7 8
 Gender: FEMALE
 Master ID: [Blank]

Answer Sheet:

Key: (A) (B) (C) (D)

01	(A) (B) (C) (D)	21	(A) (B) (C) (D)	41	(A) (B) (C) (D)	61	(A) (B) (C) (D)	81	(A) (B) (C) (D)
02	(A) (B) (C) (D)	22	(A) (B) (C) (D)	42	(A) (B) (C) (D)	62	(A) (B) (C) (D)	82	(A) (B) (C) (D)
03	(A) (B) (C) (D)	23	(A) (B) (C) (D)	43	(A) (B) (C) (D)	63	(A) (B) (C) (D)	83	(A) (B) (C) (D)
04	(A) (B) (C) (D)	24	(A) (B) (C) (D)	44	(A) (B) (C) (D)	64	(A) (B) (C) (D)	84	(A) (B) (C) (D)
05	(A) (B) (C) (D)	25	(A) (B) (C) (D)	45	(A) (B) (C) (D)	65	(A) (B) (C) (D)	85	(A) (B) (C) (D)
06	(A) (B) (C) (D)	26	(A) (B) (C) (D)	46	(A) (B) (C) (D)	66	(A) (B) (C) (D)	86	(A) (B) (C) (D)
07	(A) (B) (C) (D)	27	(A) (B) (C) (D)	47	(A) (B) (C) (D)	67	(A) (B) (C) (D)	87	(A) (B) (C) (D)
08	(A) (B) (C) (D)	28	(A) (B) (C) (D)	48	(A) (B) (C) (D)	68	(A) (B) (C) (D)	88	(A) (B) (C) (D)
09	(A) (B) (C) (D)	29	(A) (B) (C) (D)	49	(A) (B) (C) (D)	69	(A) (B) (C) (D)	89	(A) (B) (C) (D)
10	(A) (B) (C) (D)	30	(A) (B) (C) (D)	50	(A) (B) (C) (D)	70	(A) (B) (C) (D)	90	(A) (B) (C) (D)
11	(A) (B) (C) (D)	31	(A) (B) (C) (D)	51	(A) (B) (C) (D)	71	(A) (B) (C) (D)	91	(A) (B) (C) (D)
12	(A) (B) (C) (D)	32	(A) (B) (C) (D)	52	(A) (B) (C) (D)	72	(A) (B) (C) (D)	92	(A) (B) (C) (D)
13	(A) (B) (C) (D)	33	(A) (B) (C) (D)	53	(A) (B) (C) (D)	73	(A) (B) (C) (D)	93	(A) (B) (C) (D)
14	(A) (B) (C) (D)	34	(A) (B) (C) (D)	54	(A) (B) (C) (D)	74	(A) (B) (C) (D)	94	(A) (B) (C) (D)
15	(A) (B) (C) (D)	35	(A) (B) (C) (D)	55	(A) (B) (C) (D)	75	(A) (B) (C) (D)	95	(A) (B) (C) (D)
16	(A) (B) (C) (D)	36	(A) (B) (C) (D)	56	(A) (B) (C) (D)	76	(A) (B) (C) (D)	96	(A) (B) (C) (D)
17	(A) (B) (C) (D)	37	(A) (B) (C) (D)	57	(A) (B) (C) (D)	77	(A) (B) (C) (D)	97	(A) (B) (C) (D)
18	(A) (B) (C) (D)	38	(A) (B) (C) (D)	58	(A) (B) (C) (D)	78	(A) (B) (C) (D)	98	(A) (B) (C) (D)
19	(A) (B) (C) (D)	39	(A) (B) (C) (D)	59	(A) (B) (C) (D)	79	(A) (B) (C) (D)	99	(A) (B) (C) (D)
20	(A) (B) (C) (D)	40	(A) (B) (C) (D)	60	(A) (B) (C) (D)	80	(A) (B) (C) (D)	100	(A) (B) (C) (D)

Appendix Figure A.1: Photographed Answer Sheet as Uploaded (input)



FAST TRACK
Engineering Institute

FAST TRACK ENGINEERING INSTITUTE

Adwait Marg, Bagbazar Kathmandu
☎ 01-5347362, 9851358947, 9851407904

Rollno: 49920

score: 240.00

correct: 100

wrong: 0

skipped: 0

Invalid: 0

First Name ANURAJ		Middle Name KUNAR		Last Name JHA	
शिक्षापत्र नं.: रोल नं.: उम्मेदवारको नाम: उम्मेदवारको हस्ताक्षर:  उम्मेदवारको बुवाको नाम: उम्मेदवारको आमाको नाम: उम्मेदवारको हजुरबुवाको नाम: स्थायी ठेगाना, जिल्ला: गा.पा./व.पा.: पञ्चायत नं.: मिति: 2082-03-28		Roll Number 4 9 9 2 0		Gender MALE ☒ FEMALE ☐ जा. प्र. ☐ अ. प्र. ☐ अ. प्र. ☐ अ. प्र. ☐ खुला ☐ मसिलो ☐ प. छेद ☐ पहिला ☐ दोस्रो ☐ निर्देशन १. आफ्नो रोल नम्बर अंकमा कससँगै लेख्नुहोस् र सोही अंकसँग मिल्ने तलको नम्बरमा पनि चिह्न लगाउनुहोस्। २. लेखिसकेपछि उतर मेडुले वा काट्ने नगर्नुहोस्। ३. उतर चिह्न लगाउनका लागि तयारीलाई चिह्नको कालो पेन मात्र प्रयोग गर्नुहोस्। ४. उतर चिह्न लगाउँदा पेन चुरा रूपमा भर्नुहोस्। ५. यस उत्तरपुस्तिकामा कुनै पनि अनावश्यक चिह्न वा रङ काम नगर्नुहोस्। ६. प्रश्नपत्रमा चिह्नको सेट नम्बर अनुसार आफ्नो सेट नम्बर भर्नुहोस्।	
Master ID 		Incorrect Fill ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐		Correct Fill ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	

केन्द्राध्यक्षको दस्तखत



सहायक केन्द्राध्यक्षको दस्तखत

Key		A		B		C		D	
01	A	02	B	03	C	04	D	05	A
06	B	07	C	08	D	09	A	10	B
11	C	12	D	13	A	14	B	15	C
16	D	17	A	18	B	19	C	20	D
21	A	22	B	23	C	24	D	25	A
26	B	27	C	28	D	29	A	30	B
31	C	32	D	33	A	34	B	35	C
36	D	37	A	38	B	39	C	40	D
41	A	42	B	43	C	44	D	45	A
46	B	47	C	48	D	49	A	50	B
51	C	52	D	53	A	54	B	55	C
56	D	57	A	58	B	59	C	60	D
61	A	62	B	63	C	64	D	65	A
66	B	67	C	68	D	69	A	70	B
71	C	72	D	73	A	74	B	75	C
76	D	77	A	78	B	79	C	80	D
81	A	82	B	83	C	84	D	85	A
86	B	87	C	88	D	89	A	90	B
91	C	92	D	93	A	94	B	95	C
96	D	97	A	98	B	99	C	100	D

Appendix Figure A.2: Annotated Preview of a Graded Sheet (output)

Exported CSV report — excerpt of the actual omr_report.csv produced by the graded session of Section 5.1.3 (long rows truncated with "..."):

```
File_name,Rollno,Q1,Q2,Q3,Q4,Q5,Q6,Q7,Q8,Q9,Q10,Q11,Q12,Q13,Q14,Q15,Q16,Q17,
Q18,Q19,Q20,Q21,Q22 ...
page_0003 (Copy
3).png,46319,B,,C,,D,C,,,C,,C,,D,,,,,,,,,C,,,D,,,C,,,C,,B,,D,,A,C,,,A,,C,,
A,C ...
page_0002 (Copy
12).png,95878,,,,,,,,,B,,,C,,B,B,D,B,,D,,B,,,,,,,,,D,D,B,D,D,D,C,A,,A,A,,A
,,, ...
page_0002 (Copy
7).png,95878,,,,,,,,,B,,,C,,B,B,D,B,,D,,B,,,,,,,,,D,D,B,D,D,D,C,A,,A,A,,A,
,,, ...
page_0002 (Copy
3).png,95878,,,,,,,,,B,,,C,,B,B,D,B,,D,,B,,,,,,,,,D,D,B,D,D,D,C,A,,A,A,,A,
,,, ...
page_0004
(Copy).png,35891,D,B,C,B,B,B,A,B,B,C,B,B,B,A,B,C,A,D,A,C,B,A,C,B,A,B,D,C,C,B
,D,B,C,C, ...
```

Appendix B: Snippets of Major Source Code Components

Single-choice decoding (backend_core/omr_engine_fast.py)

```
@staticmethod
def _decode_single_choice(marked_bool: np.ndarray) -> np.ndarray:
    """
    returns code per row:
    -2 => INV (multiple)
    -1 => skipped (none)
    >=0 => selected index
    """
    count = marked_bool.sum(axis=1)
    sel = marked_bool.argmax(axis=1)
    code = np.full((marked_bool.shape[0],), -1, dtype=np.int32)
    code[count == 1] = sel[count == 1]
    code[count > 1] = -2
    return code
```

Integral-image fill-ratio measurement (backend_core/omr_engine_fast.py)

```
bw = self._threshold_binary01(gray)          # 0/1 ink mask (T = 130,
inverse)
integ = cv2.integral(bw, sdepth=cv2.CV_32S)  # one integral image per sheet

# Roll-number grid
roll_sum = self._ellipse_sum_integral(integ, self._roll_boxes)
roll_fill = roll_sum / np.maximum(self._roll_area, 1)
roll_marked = (roll_fill > self.cfg.fill_ratio).reshape(
    self.roll_pts.shape[0], self.roll_pts.shape[1])
roll_code = self._decode_single_choice(roll_marked)

# Answer grid
ans_sum = self._ellipse_sum_integral(integ, self._ans_boxes)
ans_fill = ans_sum / np.maximum(self._ans_area, 1)
ans_marked = (ans_fill > self.cfg.fill_ratio).reshape(
    self.ans_pts.shape[0], self.ans_pts.shape[1])
ans_code = self._decode_single_choice(ans_marked)
```

Scoring with negative marking and subject-wise totals

```
for q, subject, Marks_Correct, Negative_Percent, correct_ans in
self.answer_key:
    stu = (answers.get(q, "") or "").strip().upper()
    if stu == "":
        skipped += 1; subj_skipped[subject] += 1
        continue
    if stu == "INV":
        invalid += 1; subj_invalid[subject] += 1
        continue
    if stu == correct_ans:
        correct += 1; subj_correct[subject] += 1
        score += Marks_Correct; subj_score[subject] += Marks_Correct
    else:
        neg = Negative_Percent * Marks_Correct
        wrong += 1; subj_wrong[subject] += 1
        score -= neg; subj_score[subject] -= neg
```

Marker ordering and perspective warp (calibration.py)

```
# order the four detected marker centres: tl, tr, br, bl
rect = np.zeros((4, 2), dtype="float32")
s = pattern.sum(axis=1)
rect[0] = pattern[np.argmin(s)]      # top-left: smallest x+y
rect[2] = pattern[np.argmax(s)]      # bottom-right: largest x+y
diff = np.diff(pattern, axis=1)      # y - x
rect[1] = pattern[np.argmin(diff)]   # top-right
rect[3] = pattern[np.argmax(diff)]   # bottom-left

crop = warp_crop_gray(gray_imgn, rect, dimeninfo) # perspective warp
if mid_point_below < mid_point_upper:             # 180-degree fix
    crop = cv2.rotate(crop, cv2.ROTATE_180)
```

Starting a grading job (app.py)

```
@app.post("/run", response_model=RunResponse)
async def start_run(answers: UploadFile, template: UploadFile,
                   images: List[UploadFile] = File(...)):
    job_id = str(uuid.uuid4())
    # ... decode images, validate answers CSV and template pickle ...
    threading.Thread(
        target=run_job,
        args=(job_id, image_names, image_arrays,
              answers_payload, template_payload),
        daemon=True,
    ).start()
    return RunResponse(job_id=job_id)
```

Role-based route guarding (frontend/src/middleware.ts)

```
const GUARDS: Array<[ReturnType<typeof createRouteMatcher>, Role[]]> = [
  [createRouteMatcher(["/super-admin(.*)"]), ["SUPER_ADMIN"]],
  [createRouteMatcher(["/admin(.*)"]), ["SUPER_ADMIN", "ORG_ADMIN"]],
  [createRouteMatcher(["/student(.*)"]), ["SUPER_ADMIN", "STUDENT"]],
];

export default clerkMiddleware(async (auth, req) => {
  const guard = GUARDS.find(([matches]) => matches(req));
  if (!guard) return;

  const { userId, sessionClaims } = await auth();
  if (!userId) {
    return NextResponse.redirect(new URL("/login", req.url));
  }
  // Role travels in the Clerk session token's `metadata` claim.
  const role = sessionClaims?.metadata?.role;
  if (!role || !guard[1].includes(role)) {
    return NextResponse.redirect(new URL("/", req.url));
  }
});
```

Appendix C: Log of Visits to Supervisor

S.N.	Date	Discussion / Technical Feedback	Supervisor's Signature
------	------	------------------------------------	---------------------------

1			
2			
3			
4			
5			
6			
7			
8			